ADKV: A Low-Overhead Adaptive Delta Quantization for KV Cache in LLM Inference

Honghao Jia[1] **Zhenxing Li**[2] **Jialiang Guo**[3] **Jiacheng Gan**[2]
[1]University of Science and Technology of China
[2]Harbin Engineering University
[3]Tianjin University
jiahonghao@mail.ustc.edu.cn

Abstract

The KV cache is a major memory bottleneck in LLM inference, especially under large batch sizes and long contexts. While quantization is a promising solution, existing static per-group quantization methods are constrained by a trade-off between metadata overhead and accuracy: increasing the group size reduces group-wise metadata (e.g., zero points, scaling factors, and outlier indices), but exacerbates quantization error under the sinusoidal-like temporal drift of post-RoPE outlier channels; conversely, using smaller groups improves accuracy but incurs substantial metadata overhead. In this paper, we propose **ADKV**, a low-overhead adaptive delta quantization framework for KV cache in LLM inference, which leverages the temporal structure of the KV cache. ADKV tracks channel-wise zero points and scaling factors online via EMA, adapting to the sinusoidal-like temporal drift, thereby eliminating the rigidity of static per-group quantization. This adaptive mechanism further enables delta coding to embed metadata implicitly into the quantized sequence, decoupling metadata overhead from context length. To minimize the MSE of attention outputs, we design an SGD-based calibration algorithm that models ADKV as a recurrent computation graph. Experiments on generation and long-context benchmarks demonstrate that ADKV reduces total KV cache memory by 5.87–19.99% and quantization metadata memory by 99.76–99.94% compared to the static per-group quantization baselines, while maintaining comparable accuracy. On an NVIDIA L40 (48GB) GPU, a custom fused ADKV-GEMV CUDA kernel optimized for memory-bound autoregressive decoding achieves a $2\times$ larger maximum feasible batch size, and $1.53\times$ end-to-end throughput gain at a batch size of 128 over the baselines. Code: github.com/LoveLonelyTime/ADKV

1 Introduction

Large language models (LLMs) achieve strong performance across diverse natural language processing (NLP) tasks [45, 39, 20, 14], but their deployment remains computationally expensive [28]. In batch inference settings [46], the key-value (KV) cache, which stores intermediate attention activations to avoid redundant computation, becomes a major memory bottleneck, especially for long-context inputs [30]. For instance, the KV cache in DeepSeek-V3 can reach 4.2TB under a 128K-token and 512-batch setting, exceeding the model parameters themselves (1.34TB in FP16) [44, 9]. As a result, efficient KV cache compression without accuracy loss becomes a critical research challenge [34, 43, 1, 21, 25, 16, 36, 23].

Among existing compression methods, quantization is particularly practical: it directly reduces KV cache bit-width without modifying the attention mechanism, achieving a 2–$4\times$ memory footprint reduction with negligible accuracy loss [18, 22]. However, existing static per-group quantization methods such as KIVI [25] and KVQuant [18] rely on fixed-window statistics, incurring non-

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

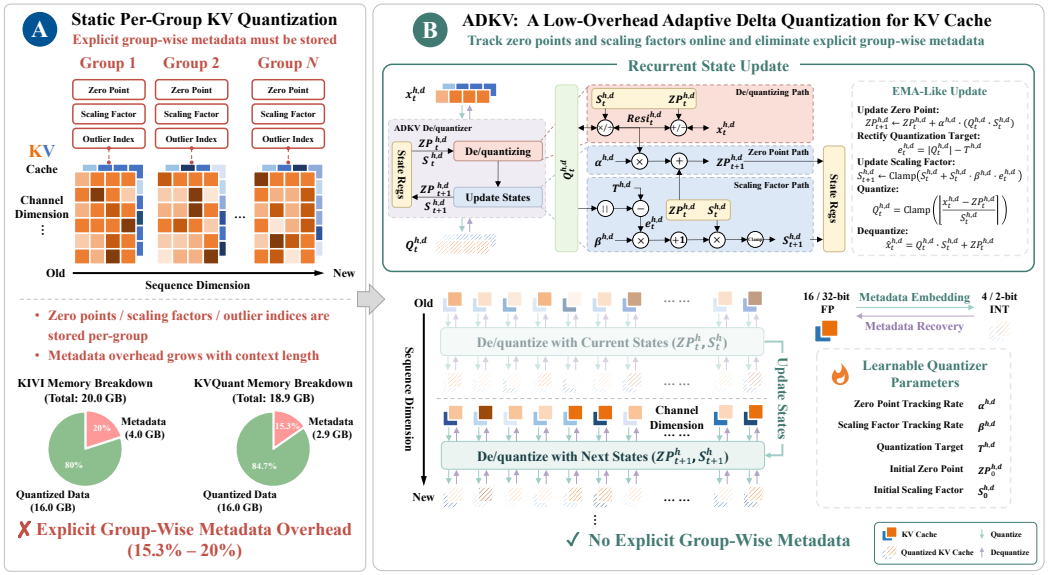


Figure 1: **(A)** Overall architecture of static per-group KV quantization. The memory breakdown of KIVI and KVQuant on Llama-2 7B shows that metadata accounts for 20% of the quantized KV cache in KIVI and 15.3% in KVQuant. **(B)** Overall architecture of ADKV. ADKV employs an EMA-based per-channel quantizer to embed metadata into the quantized sequence, thereby reducing metadata storage and bandwidth overhead.

negligible metadata overhead (15.3–20%) for storing zero points, scaling factors, and outlier indices, as illustrated in Figure 1 (A). Extending the statistical range reduces the metadata overhead but degrades accuracy due to distributional shifts in long contexts. As shown in Figure 2 Middle, influenced by the rotational transformation of RoPE [35], certain outlier channels exhibit a sinusoidal-like temporal drift. This drift further complicates the quantization of key activation channels. Static per-group quantization methods assume that these channels exhibit stable dynamic ranges that effectively determine the metadata of each group [18], but this assumption is challenged by the temporal drift.

We instead observe that the temporal drift induced by the sequential generation of KV entries exhibits an inherent temporal structure that can be leveraged to reduce the metadata overhead. Along the sequence dimension, zero points and scaling factors exhibit temporal coherence rather than independent fluctuations. Therefore, the metadata at each decoding step can be recurrently updated from that of the previous step, effectively tracking the metadata states across decoding steps and eliminating the need to explicitly store metadata for every group.

Motivated by the observations and limitations of these existing methods, we propose ADKV, a low-overhead adaptive delta quantization framework designed to reduce the metadata overhead, while mitigating accuracy degradation over extended statistical ranges. As illustrated in Figure 1 (B), the ADKV quantizer tracks and recurrently updates channel-wise zero points and scaling factors token-by-token using exponential moving average (EMA), while the ADKV dequantizer reconstructs them using the same EMA process. By exploiting redundancy in the temporal structure of the KV cache and embedding metadata implicitly into the quantized sequence, ADKV significantly reduces the metadata overhead by making it independent of context length. Furthermore, the token-level updates of zero points and scaling factors employed by ADKV overcome the rigidity of static per-group quantization, thereby helping preserve quantization accuracy.

We make the following contributions:

- **Analysis of the Temporal Structure in the KV Cache.** We observe and formalize the inherent temporal structure of the KV cache.
- **Adaptive Delta Quantization via EMA.** We employ EMA to recurrently track and update zero points and scaling factors online, thereby improving robustness to distributional shifts.

- **Ultra-Low Metadata Overhead via Adaptive Delta Coding.** We decouple the metadata overhead from context length by embedding metadata implicitly into the quantized sequence via adaptive delta coding, reducing total KV cache memory by 5.87–19.99% and quantization metadata memory by 99.76–99.94% compared to the static per-group quantization baselines.
- **SGD-Based Calibration Algorithm.** We develop an SGD-based calibration algorithm that optimizes the quantizer parameters by minimizing the mean squared error (MSE) of attention outputs, which enables attention-aware calibration.
- **High-Performance Fused ADKV-GEMV CUDA Kernel.** We design an efficient CUDA kernel for the ADKV quantization and dequantization pipeline. On an NVIDIA L40 (48GB) GPU, the kernel achieves a $2\times$ larger maximum feasible batch size, and $1.53\times$ end-to-end throughput gain at a batch size of 128 over the baselines.

2 Background

2.1 LLM Inference

For decoder-only Transformer LLMs, inference consists of two phases: prefill and decoding. In the prefill phase, the model processes the input prompt and stores the resulting key and value activations in the KV cache. In the decoding phase, tokens are produced autoregressively, and each token attends to all previously cached key and value activations, avoiding redundant computation and enabling linear complexity with respect to sequence length.

However, the KV cache incurs substantial memory overhead, especially for large models and long contexts. Its memory footprint scales as $\mathcal{O}(L \times T \times H \times d_h \times 2 \times b)$, where L is the number of layers, T the sequence length, H the number of attention heads, d_h the per-head hidden dimension, and b the number of bytes per element (e.g., 2 for FP16). This linear scaling makes the KV cache a dominant memory bottleneck, motivating compression and quantization methods to reduce its footprint while preserving accuracy.

2.2 KV Cache Quantization

KV cache quantization provides a more principled pathway for reducing the memory footprint, as it directly reduces the bit-width required to represent each stored key and value activation. To the best of our knowledge, the most common formulation for KV cache quantization is round-to-nearest (RTN) quantization, which can be expressed as:

$$Q(X) = \mathrm{Clamp}\left(\left\lfloor \frac{X - z_x}{s_x} \right\rceil, 0, 2^B - 1\right), \quad X' = Q(X) \cdot s_x + z_x \tag{1}$$

where B denotes the bit-width, $z_x = \min(X)$ is the zero point, and $s_x = (\max(X) - \min(X))/(2^B - 1)$ is the scaling factor. Here, $\mathrm{Clamp}(\cdot)$ and $\lfloor \cdot \rceil$ denote the clipping and rounding operators, respectively. Due to the streaming generation of the KV cache, the determination of zero points and scaling factors allows for different design choices. Prior work commonly adopts either per-token or per-channel quantization strategies for KV cache quantization [25, 18]. In particular, keys are often quantized per channel, motivated by the observation that certain key channels (i.e., outlier channels) exhibit significantly larger magnitude variance, while values are typically quantized per token since they tend to have more uniform distributions without pronounced outlier channels [25].

3 Methodology

Motivated by the limitations discussed in Section 1, we first revisit the temporal structure of the KV cache during autoregressive inference. Building upon this analysis, we propose ADKV, a low-overhead adaptive delta quantization framework designed to address both the rigidity of static per-group quantization and associated metadata overhead, while maintaining computational efficiency on modern GPU architectures.

3.1 Motivations

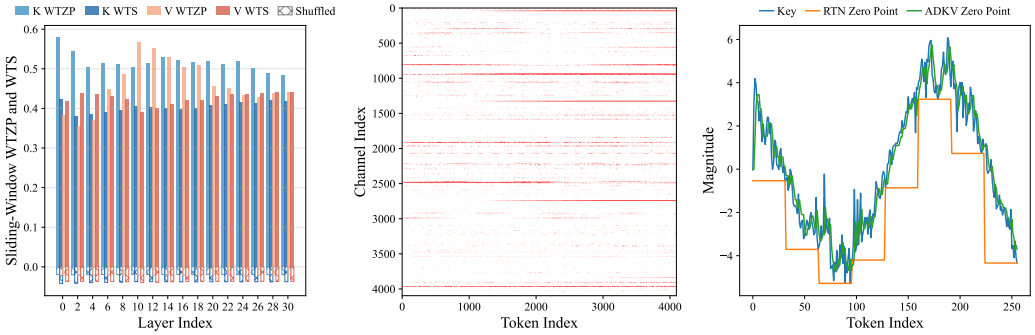


Figure 2: **Left:** Optimal WTZP and WTS ($w = 8, s = 4$) across layers of Llama-2 7B on the Wikipedia dataset [11]. Both keys and values exhibit significantly larger WTZP and WTS than those of the randomly shuffled tokens. **Middle:** Distribution of keys at layer 12, where outlier channels (red lines) show sinusoidal-like dynamics rather than stable dynamic ranges. **Right:** Comparison of zero point estimation between ADKV and RTN (Dim 24, Token 0–256). ADKV maintains a recurrently updated mean, whereas RTN uses the minimum value within a sliding window (e.g., 32 tokens).

In the decoding phase of decoder-only LLMs, the KV cache grows in a sequential, time-ordered manner, exhibiting an inherent temporal structure that static per-group quantization fails to exploit. To formalize this intuition, we introduce **temporal trackability**, which measures the trackability of zero points and scaling factors along the sequence dimension via an EMA tracker.

Given the KV cache $X \in \mathbb{R}^{T\times H\times d_h}$ at a layer, we compute sliding-window local means $\bar{x}_t^{h,d} = \frac{1}{w}\sum_{i=t}^{t+w-1} x_i^{h,d}$ and deviations $m_t^{h,d} = \frac{1}{w}\sum_{i=t}^{t+w-1} \left| x_i^{h,d} - \bar{x}_t^{h,d} \right|, t \in \{0, s, 2s, \dots\}$ with a window size w and stride s. We then simulate an EMA tracker $z_t^{h,d} = z_{t-1}^{h,d} + \alpha^{h,d} \cdot r_t^{h,d}$, where the residual $r_t^{h,d}$ is defined as either $\bar{x}_t^{h,d} - z_{t-1}^{h,d}$ or $m_t^{h,d} - z_{t-1}^{h,d}$. We define two trackability metrics:

$$WTZP^{h,d} = 1 - \frac{\mathrm{Var}[r_t^{h,d}]}{\mathrm{Var}[\bar{x}_t^{h,d}] + \epsilon}, \qquad WTS^{h,d} = 1 - \frac{\mathrm{Var}[r_t^{h,d}]}{\mathrm{Var}[m_t^{h,d}] + \epsilon} \tag{2}$$

where WTZP measures the zero point trackability and WTS measures the scaling factor trackability. Values close to 1 indicate an EMA-trackable sequence, whereas lower values indicate the opposite. As shown in Figure 2 Left, the optimal WTZP and WTS of the KV cache are consistently larger than those of a randomly shuffled sequence across all layers of Llama-2 7B, confirming that both zero points and scaling factors exhibit the stable temporal structure well-suited for adaptive delta quantization.

3.2 ADKV: Adaptive Delta Quantization for KV Cache

As analyzed in Section 3.1, zero points and scaling factors can be effectively tracked per channel over the sequence dimension. To effectively track and compress the trackable components, we propose ADKV, an adaptive delta quantization method for KV cache. Unlike static per-group quantization, the ADKV quantizer maintains two state variables for adaptive delta representation: the current zero point and scaling factor.

Quantization Phase: When the new t-th token is appended, the per-channel quantizer first computes a residual by subtracting the current zero point from the channel value to be quantized:

$$Resi_t^{h,d} = x_t^{h,d} - ZP_t^{h,d} \tag{3}$$

where $ZP_t^{h,d}$ is one of the two state variables for the ADKV quantizer, which is responsible for tracking the zero point. Then, the quantizer scales and quantizes the residual by the current scaling factor:

$$Q_t^{h,d} = \text{Clamp}\left(\left\lfloor \frac{Resi_t^{h,d}}{S_t^{h,d}} \right\rceil, Q_\text{L}, Q_\text{U}\right) \tag{4}$$

where $S_t^{h,d}$ is another state variable for the ADKV quantizer, which is responsible for tracking the adaptive scaling factor. Q_L and Q_U are the lower and upper bounds for the clipping function $\text{Clamp}(\cdot)$. For ADKV, we simply use the RTN function $\lfloor \cdot \rceil$ for signed integers.

After the quantization phase, the quantized value $Q_t^{h,d}$ is packed and stored in the KV cache. Unlike static per-group quantization, which requires storing the zero point and scaling factor associated with $Q_t^{h,d}$, ADKV has a significant advantage in that it does not need to store either the zero point or scaling factor. This is because the zero point and scaling factor are implicitly embedded into the quantized sequence and can be recovered during the dequantization phase.

Update Phase: In the update phase, the ADKV quantizer updates all state variables it maintains. As discussed in Section 3.1, the ADKV quantizer also adopts EMA to update the current zero point and scaling factor:

$$\begin{aligned} ZP_{t+1}^{h,d} &\leftarrow ZP_t^{h,d} + \alpha^{h,d} \cdot \left(Q_t^{h,d} \cdot S_t^{h,d}\right) \\ e_t^{h,d} &= \left|Q_t^{h,d}\right| - T^{h,d} \\ S_{t+1}^{h,d} &\leftarrow \text{Clamp}(S_t^{h,d} + S_t^{h,d} \cdot \beta^{h,d} \cdot e_t^{h,d}, S_\text{min}, S_\text{max}) \end{aligned} \tag{5}$$

where $\alpha^{h,d} \in (0,1)$, $\beta^{h,d}, T^{h,d} > 0$ are constant parameters for channel (h,d). For updating $ZP_{t+1}^{h,d}$, the ADKV quantizer multiplies the reconstructed value $Q_t^{h,d} \cdot S_t^{h,d}$ by $\alpha^{h,d}$, where $\alpha^{h,d}$ controls the EMA tracking rate of the zero point. $\alpha^{h,d}$ can also be interpreted as the weighting factor controlling the contribution of the newly appended value $Q_t^{h,d} \cdot S_t^{h,d}$ to $ZP_{t+1}^{h,d}$. A larger $\alpha^{h,d}$ increases the responsiveness of zero point tracking for channel (h,d), whereas a smaller $\alpha^{h,d}$ results in slower adaptation to changes in the zero point.

For tracking the scaling factor, we need a more cautious strategy, because the ADKV quantizer is very sensitive to the scaling factor. A value to be quantized divided by an excessively small scaling factor overloads the quantizer, while a value divided by an excessively large scaling factor underloads it. Both situations will result in a significant decrease in quantizer accuracy. In Equation (5), we aim for the quantized value to remain within the optimal operating range of the quantizer. Therefore, we define the quantization target $T^{h,d}$ that the ADKV quantizer aims to maintain. The ADKV quantizer updates $S_{t+1}^{h,d}$ additively in the log domain according to $e_t^{h,d}$, which represents the current error between the absolute magnitude $\left|Q_t^{h,d}\right|$ and $T^{h,d}$:

$$S_{t+1}^{h,d} \leftarrow e^{\log S_t^{h,d} + \beta^{h,d} \cdot e_t^{h,d}} = S_t^{h,d} \cdot e^{\beta^{h,d} \cdot e_t^{h,d}} \tag{6}$$

where $\beta^{h,d} > 0$, yielding the exponential update $e^{\beta^{h,d} \cdot e_t^{h,d}}$. To ensure stable scaling factor updates, we choose a sufficiently small update gain such that $|\beta^{h,d} e_t^{h,d}| \ll 1$. We can then apply the first-order Taylor expansion $e^{\beta^{h,d} \cdot e_t^{h,d}} \approx 1 + \beta^{h,d} \cdot e_t^{h,d}$, which yields the hardware-efficient scaling factor update in Equation (5). Finally, to prevent $S_t^{h,d}$ from inadvertently becoming excessively large or small, we clamp it within a safe range $[S_\text{min}, S_\text{max}]$.

Dequantization Phase: In fact, the dequantization phase is a subroutine in the quantization phase. The ADKV dequantizer maintains the same state variables by adopting the update strategy in Equation (5). One main difference is reconstructing the original value using the current zero point and scaling factor:

$$\hat{x}_t^{h,d} = Q_t^{h,d} \cdot S_t^{h,d} + ZP_t^{h,d} \tag{7}$$

The quantization phase mirrors the dequantization phase. Based on this design, we can omit explicitly restoring the zero point and scaling factor, which are implicitly embedded into the quantized sequence. As shown in Figure 2 Right, ADKV can accurately recover the zero point from the quantized sequence without relying on any grouped metadata, requiring only the initial states and per-channel constant parameters. In contrast, methods such as KIVI and KVQuant that employ static per-group quantization rely on coarse-grained zero points. This highlights the limitation of static per-group quantization in capturing fine-grained, dynamically evolving quantization statistics.

3.3 Learnable Quantizer Parameter

The ADKV quantizer requires three per-channel constant parameters: $\alpha^{h,d}$, $\beta^{h,d}$, and $T^{h,d}$. $\alpha^{h,d}$ and $\beta^{h,d}$ control the tracking rates of the zero point and scaling factor, respectively. $T^{h,d}$ is used to retain the quantized values within the optimal operating range of the quantizer. In addition, the initial states of the quantizer, $ZP_0^{h,d}$ and $S_0^{h,d}$, determine the convergence rate of the system. The selection of the above parameters directly determines the accuracy performance of the ADKV quantizer. We employ the learnable quantizer parameter (LQP) strategy with stochastic gradient descent (SGD).

Main Idea: We observe that the ADKV quantizer and dequantizer together form a recurrent computation graph, as illustrated in Figure 1 (B), enabling the use of recurrent optimization techniques for automatic parameter optimization. We extract a calibration set by sampling a subset of sequences from the original training dataset, specifically by selecting contiguous token sequences along with their corresponding query, key, and value activations, which preserve the temporal dynamics and statistical properties of the full dataset. This calibration set serves as a lightweight yet representative proxy, enabling efficient optimization of ADKV parameters without incurring the cost of full-scale retraining.

Prior work has shown that optimizing only the reconstruction of values is insufficient to preserve accurate operator outputs, motivating the incorporation of the operator into the quantization process [12, 25]. Accordingly, we design a training objective that preserves attention accuracy through attention-aware calibration. For each sample i, we first calculate the reference attention scores and attention outputs using the original query, key, and value activations. Then, the quantized keys and values produced by ADKV are fed back into the same attention operator to obtain the reconstructed outputs. The primary loss consists of an MSE term:

$$\mathcal{L}_{\text{attn}}^{(i)} = \frac{1}{H}\sum_{h=0}^{H-1}\frac{1}{|\Omega^{(i)}|}\left\|\text{Softmax}\left(\frac{Q_h^{(i)}\,\hat{K}_h^{(i)\top}}{\sqrt{d_h}}\right)\hat{V}_h^{(i)} - \text{Softmax}\left(\frac{Q_h^{(i)}\,K_h^{(i)\top}}{\sqrt{d_h}}\right)V_h^{(i)}\right\|_F^2 \tag{8}$$

where $K_h^{(i)}$ and $V_h^{(i)}$ denote the original KV pairs, and $\hat{K}_h^{(i)}$ and $\hat{V}_h^{(i)}$ are their counterparts reconstructed from the quantized KV pairs. $|\Omega^{(i)}|$ denotes the total number of values in the attention output, given by $T_{\text{q}} \cdot d_h$. This formulation explicitly enables the ADKV quantizer to adapt to and be aware of the attention operator, rather than solely minimizing the MSE between the original and reconstructed KV pairs.

To ensure stability in calibration and avoid divergence in the dynamic update process, we impose explicit constraints on vital parameters through bounded nonlinearities. For example, the scaling factor tracking rate $\beta^{h,d}$ is reparameterized using a sigmoid function with a scaling factor to limit its magnitude. Appendix F provides a detailed description of ADKV calibration, and Appendix J reports ablation studies isolating the contributions of each component and of calibration.

Gradient Analysis: Like most quantization-aware methods, ADKV also needs to estimate the gradients of the clipping and rounding functions. ADKV employs the straight-through estimator (STE) [3] to approximate the gradient of the rounding function. In addition, for the scaling factor $S^{h,d}$, we adopt learned step size quantization (LSQ) to estimate its gradient [10]. All parameter gradients can be obtained by differentiating Equation (5):

Theorem 1. *Let $\partial\mathcal{L}_{attn}/\partial\hat{V}_{h,t,d}$ or $\partial\mathcal{L}_{attn}/\partial\hat{K}_{h,t,d}$ be the upstream gradient from Equation* (8)*. Taking $\partial\mathcal{L}_{attn}/\partial\hat{K}_{h,t,d}$ (i.e., the K cache) as an example, the recurrent gradients of ADKV quantizer parameters are given:*

$$
\begin{aligned}
\frac{\partial \mathcal{L}_{attn}}{\partial ZP_t^{h,d}} &= \frac{\partial \mathcal{L}_{attn}}{\partial \hat{K}_{h,t,d}}\left(1 - \mathbf{1}_t^{h,d}\right) \; + \; \frac{\partial \mathcal{L}_{attn}}{\partial ZP_{t+1}^{h,d}}\left(1 - \alpha^{h,d}\mathbf{1}_t^{h,d}\right) \\
\frac{\partial \mathcal{L}_{attn}}{\partial S_t^{h,d}} &= \frac{\partial \mathcal{L}_{attn}}{\partial \widehat{Resi}_t^{h,d}} \cdot \frac{1}{Q_U} \cdot \mathrm{LSQ}(Resi_t^{h,d}, S_t^{h,d}, \widehat{Resi}_t^{h,d}) \; + \; \frac{\partial \mathcal{L}_{attn}}{\partial S_{t+1}^{h,d}} \cdot \mathcal{A}_t^{h,d} \cdot \left(1 + \beta^{h,d} e_t^{h,d}\right) \\
\frac{\partial \mathcal{L}_{attn}}{\partial \alpha^{h,d}} &= \sum_{t=0}^{k-1} \frac{\partial \mathcal{L}_{attn}}{\partial ZP_{t+1}^{h,d}} \widehat{Resi}_t^{h,d} \\
\frac{\partial \mathcal{L}_{attn}}{\partial \beta^{h,d}} &= \sum_{t=0}^{k-1} \frac{\partial \mathcal{L}_{attn}}{\partial S_{t+1}^{h,d}} \cdot \mathcal{A}_t^{h,d} \cdot S_t^{h,d} \cdot e_t^{h,d} \\
\frac{\partial \mathcal{L}_{attn}}{\partial T^{h,d}} &= -\beta^{h,d} \sum_{t=0}^{k-1} \frac{\partial \mathcal{L}_{attn}}{\partial S_{t+1}^{h,d}} \cdot \mathcal{A}_t^{h,d} \cdot S_t^{h,d}
\end{aligned}
\tag{9}
$$

Gradients at node $\widehat{Resi}_t^{h,d}$ are accumulated from all its successors:

$$
\frac{\partial \mathcal{L}_{attn}}{\partial \widehat{Resi}_t^{h,d}} = \frac{\partial \mathcal{L}_{attn}}{\partial \hat{K}_{h,t,d}} + \alpha^{h,d} \frac{\partial \mathcal{L}_{attn}}{\partial ZP_{t+1}^{h,d}} \tag{10}
$$

where $\mathrm{LSQ}(\cdot)$ *is the LSQ gradient estimate function in Equation* (19)*,* $\mathbf{1}_t^{h,d} = \mathbf{1}[Q_L \leq Resi_t^{h,d}/S_t^{h,d} \leq Q_U]$*, and* $\mathcal{A}_t^{h,d} = \mathbf{1}[S_{min} \leq S_t^{h,d}(1 + \beta^{h,d} e_t^{h,d}) \leq S_{max}]$*.*

Proof. See Appendix D. □

3.4 Kernel Implementation

To efficiently support the ADKV quantization and dequantization pipeline and enable matrix-vector multiplication directly on the quantized KV cache, we implement a fused ADKV-GEMV CUDA kernel. During the decoding phase, the kernel loads the quantized KV cache along with per-channel quantizer parameters and initial states into GPU registers. Then, the kernel sequentially dequantizes the tokens and performs the corresponding dot-product operations. Before dequantizing the next token, the kernel updates the current zero point and scaling factor in place. Since all parameters and states are stored in registers, group-wise metadata accesses are eliminated during execution, thereby reducing runtime metadata overhead. We provide a detailed description of the kernel implementation in Appendix H.

4 Experiments

4.1 Generation and Long-Context Accuracy

Generation Tasks: We evaluate ADKV on CoQA [31], TruthfulQA [24], and GSM8K [7], covering conversational QA, factual reasoning, and mathematical reasoning. Evaluation is conducted using the `lm-evaluation-harness` framework [13]. We compare ADKV (4-bit) against FP16, per-channel RTN (4-bit), and KIVI-128g (4-bit) on Llama-2 7B, Llama-2 13B [38], Falcon3 7B [37], and Mistral 7B [19]. To balance metadata overhead and quantization accuracy, we set the KIVI group size to 128 tokens, a commonly adopted sweet spot that achieves a favorable trade-off without incurring excessive metadata cost or noticeable accuracy degradation.

As shown in Table 1, ADKV consistently achieves competitive or superior results relative to the baseline methods under this setting. RTN incurs the most pronounced degradation in accuracy, highlighting the limitations of naive per-channel quantization in preserving reasoning capabilities. For example, RTN exhibits an average point drop of 3.98 on Llama-2 7B. In contrast, ADKV substantially narrows this gap by leveraging the temporal structure of the KV cache during quantization. Across all evaluated generation tasks, ADKV maintains near-parity with the FP16 baseline (< 1 point drop), and

even outperforms KIVI-128g, which exhibits a slight degradation under static per-group quantization with long statistical ranges.

Table 1: Generation Accuracy Results

Model	Method	CoQA ↑	TruthfulQA ↑	GSM8K ↑	Avg. ↑
Llama-2 7B	FP16	64.37	32.93	13.57	36.96
	RTN	59.87	28.52	10.54	32.98
	KIVI-128g	**64.52**	31.95	12.70	36.39
	ADKV (Ours)	63.93	**34.64**	**13.50**	**37.36**
Llama-2 13B	FP16	66.26	30.60	23.43	40.10
	RTN	65.62	28.15	16.22	36.66
	KIVI-128g	66.02	30.35	23.17	39.85
	ADKV (Ours)	**67.75**	**31.21**	**23.35**	**40.77**
Falcon3 7B	FP16	65.65	45.65	76.04	62.45
	RTN	65.95	43.00	74.22	61.06
	KIVI-128g	66.10	44.21	75.89	62.07
	ADKV (Ours)	**66.37**	**44.43**	**76.02**	**62.27**
Mistral 7B	FP16	68.42	40.64	38.21	49.09
	RTN	65.45	39.78	32.22	45.82
	KIVI-128g	67.83	40.64	**36.16**	48.21
	ADKV (Ours)	**68.17**	**40.76**	35.78	**48.24**

Long-Context Tasks: We further evaluate ADKV under long-context settings. Table 2 presents the results on LongBench [2] for Llama-2 7B 32K [8], Llama-3 8B 1048K [29], Llama-2 13B, Falcon3 7B, and Mistral 7B. LongBench comprises a diverse suite of long-context benchmarks, including question answering, summarization, and few-shot learning tasks, providing a comprehensive evaluation of models under long-context settings.

As shown in Table 2, ADKV maintains accuracy close to the FP16 baseline across most tasks and models, with average score gaps within 1.02 points in all cases. In contrast, RTN exhibits substantial degradation across several models, with average drops of 6.85 and 5.70 points on Llama-2 7B 32K and Llama-2 13B, respectively. This degradation is particularly pronounced on TriviaQA and LCC, where long-context reasoning is critical. KIVI-128g partially recovers this gap but still falls behind ADKV across all five models. This consistent advantage suggests that the adaptive delta quantization employed by ADKV effectively mitigates error accumulation under long-context settings.

Table 2: Long-Context Accuracy Results

Model	Method	Qasper ↑	QMSum ↑	TREC ↑	TriviaQA ↑	MultiNews ↑	SAMSum ↑	LCC ↑	RepoB-P ↑	Avg. ↑
Llama-2 7B 32K	FP16	10.53	17.18	71.00	87.79	21.96	43.90	62.08	59.86	46.79
	RTN	10.26	14.48	68.50	75.08	19.55	34.81	49.91	46.93	39.94
	KIVI-128g	10.14	17.13	71.00	**87.62**	**21.62**	43.97	61.22	58.50	46.40
	ADKV (Ours)	**10.35**	**17.16**	**71.00**	87.49	21.32	**43.98**	**61.62**	**58.98**	**46.49**
Llama-3 8B 1048K	FP16	27.84	25.13	71.00	87.70	28.38	42.42	38.34	38.45	44.91
	RTN	26.45	23.91	70.00	84.26	28.01	41.14	30.84	33.37	42.25
	KIVI-128g	27.16	24.53	71.00	**87.13**	28.04	**41.72**	35.21	35.14	43.74
	ADKV (Ours)	**27.52**	**24.63**	**72.00**	86.33	**28.31**	41.69	**35.32**	**35.35**	**43.89**
Llama-2 13B	FP16	9.28	21.45	70.00	87.37	15.96	43.47	66.64	56.45	46.33
	RTN	6.80	17.57	61.00	72.12	13.40	41.26	61.41	51.46	40.63
	KIVI-128g	8.73	21.23	69.00	86.20	15.21	**44.09**	**66.20**	57.06	45.97
	ADKV (Ours)	**8.81**	**21.48**	**70.00**	**87.17**	**15.53**	43.51	65.94	**57.10**	**46.19**
Falcon3 7B	FP16	12.33	23.37	77.00	89.38	23.49	43.77	68.29	62.10	49.97
	RTN	12.11	23.19	73.50	89.18	22.34	41.74	66.97	61.68	48.84
	KIVI-128g	**12.19**	**23.45**	76.00	89.25	22.22	43.22	67.45	**62.59**	49.55
	ADKV (Ours)	12.03	22.81	**76.50**	**89.40**	**22.72**	**43.25**	**67.79**	62.12	**49.58**
Mistral 7B	FP16	8.07	19.16	66.50	89.21	13.83	41.24	69.31	61.48	46.10
	RTN	7.43	**19.34**	64.50	86.63	13.22	40.87	66.66	58.44	44.64
	KIVI-128g	7.55	19.03	65.50	**89.10**	13.62	**40.95**	69.36	60.21	45.67
	ADKV (Ours)	**8.03**	19.25	**65.50**	88.94	**14.24**	39.76	**69.64**	**60.27**	**45.70**

Additional accuracy evaluations, including matched-bit-width baselines and 2-bit results, are provided in Appendix K.

4.2 Memory Savings

We evaluate the memory footprint of ADKV (4-bit), KIVI (4-bit, default group size $g = 32$), KIVI-128g (4-bit), MXFP4 (Microscaling FP4) [32], and KVQuant (4-bit). As shown in Figure 3, methods based on static per-group quantization exhibit a significantly higher memory footprint. This overhead arises from the need to store metadata for each group. As the sequence length increases, the number of groups grows proportionally, leading to a linear accumulation of metadata and, consequently, substantial memory overhead relative to the quantized KV cache.

ADKV maintains a constant metadata footprint by exploiting temporal redundancy in the KV cache and reconstructing metadata online from the quantized sequence, instead of storing group-wise metadata. Its memory footprint depends only on model configuration (layers, heads, and hidden dimension) and is independent of sequence length. This removes the main overhead in static per-group quantization, reducing total KV cache memory by 19.99% and 5.87% and quantization metadata memory by 99.94% and 99.76% compared with KIVI and KIVI-128g, respectively, under large-batch, long-sequence settings (e.g., Llama-2 7B, batch size 32, sequence length 4096, and hidden dimension 4096). Appendix G provides a detailed breakdown of the ADKV memory footprint.

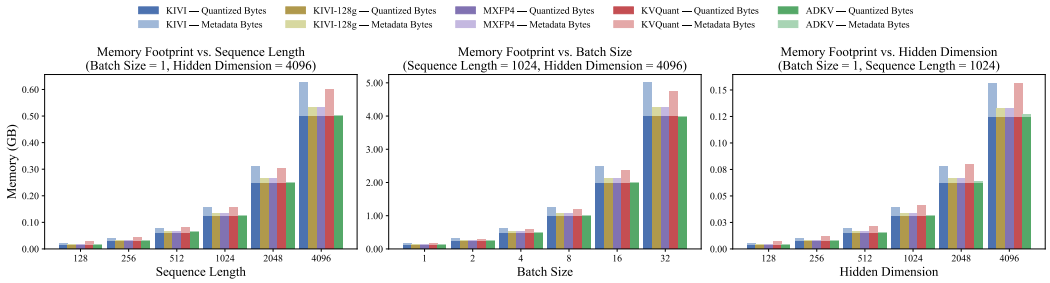


Figure 3: Memory footprint comparison of KIVI (4-bit), KIVI-128g (4-bit), MXFP4, ADKV (4-bit), and KVQuant (4-bit) on Llama-2 7B.

4.3 End-to-End Throughput

We evaluate the end-to-end throughput, including both prefill and decoding, of our fused ADKV-GEMV kernel against the FP16 GEMV and KIVI-GEMV kernels on an NVIDIA L40 (48GB) GPU with Llama-2 7B. As shown in Figure 4, under the 48GB memory constraint, ADKV supports a maximum feasible batch size of 256 without running into out-of-memory (OOM) errors, which is $2\times$ larger than KIVI and $4\times$ larger than FP16, attributable to its ultra-low metadata overhead. ADKV achieves a $1.96\times$ end-to-end throughput gain over the FP16 baseline at a batch size of 64, and $1.53\times$ gain over KIVI-128g at a batch size of 128.

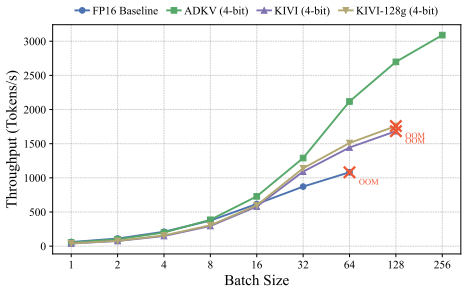


Figure 4: End-to-end throughput comparison of FP16, KIVI (4-bit), KIVI-128g (4-bit), and ADKV (4-bit) on Llama-2 7B.

The fused ADKV-GEMV kernel inevitably introduces inherent sequential dependencies, which may have a more noticeable impact in small-scale inference settings (e.g., batch size 1 with < 128 tokens). However, we observe that the FP16 GEMV baseline is predominantly memory-bound. As a result, the latency associated with the sequential dependencies is largely hidden by memory accesses in practical deployments, particularly in industrial LLM inference regimes (e.g., batch sizes > 128 and sequence lengths > 1K tokens), and the kernel can even benefit from the reduced memory bandwidth demand of 4-bit representations. For paged or distributed KV cache serving, ADKV can preserve attention parallelism by storing lightweight boundary state checkpoints per shard. Appendix I details the checkpoint design.

5 Conclusion

In this work, we propose ADKV, a low-overhead adaptive delta quantization framework for KV cache that mitigates the accuracy-metadata trade-off in static per-group quantization. Guided by temporal trackability metrics (WTZP and WTS), ADKV uses EMA-based per-channel tracking, embeds metadata into the quantized sequence via adaptive delta coding, and optimizes parameters with an SGD-based, attention-aware calibration. Experiments on generation and long-context benchmarks show that ADKV reduces total KV cache memory by 5.87–19.99% and quantization metadata memory by 99.76–99.94% compared to the static per-group quantization baselines, while maintaining comparable accuracy. A fused ADKV-GEMV CUDA kernel further achieves a $2\times$ larger maximum feasible batch size and $1.53\times$ throughput gain over the baselines.

6 Acknowledgements

This work was supported by the Institute of Computing Technology, Chinese Academy of Sciences. We thank our coauthors for their valuable contributions and the reviewers for their constructive feedback.

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[2] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. Longbench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024.

[3] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

[4] W. R. Bennett. Spectra of quantized signals. *The Bell System Technical Journal*, 1948.

[5] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.

[6] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

[7] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[8] Together Computer. Llama-2-7b-32k, 2023. URL https://huggingface.co/togethercomputer/LLaMA-2-7B-32K.

[9] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2025.

[10] Steven K. Esser, Jeffrey L. McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S. Modha. Learned step size quantization. *arXiv preprint arXiv:1902.08153*, 2020.

[11] Wikimedia Foundation. Wikimedia downloads, 2024. URL https://dumps.wikimedia.org.

[12] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2023.

[13] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. The language model evaluation harness, 2024. URL https://zenodo.org/records/12608602.

[14] Muhammad Usman Hadi, Qasem Al Tashi, Rizwan Qureshi, Abbas Shah, Amgad Muneer, Muhammad Irfan, Anas Zafar, Muhammad Bilal Shaikh, Naveed Akhtar, Syed Zohaib Hassan, Maged Shoman, Jia Wu, Seyedali Mirjalili, and Mubarak Shah. A survey on large language models: Applications, challenges, limitations, and practical usage. *TechRxiv*, 2023. doi: 10.36227/techrxiv.23589741.v1.

[15] Insu Han, Praneeth Kacham, Amin Karbasi, Vahab Mirrokni, and Amir Zandieh. Polarquant: Quantizing kv caches with polar transformation. *arXiv preprint arXiv:2502.02617*, 2025.

[16] Jitai Hao, Yuke Zhu, Tian Wang, Jun Yu, Xin Xin, Bo Zheng, Zhaochun Ren, and Sheng Guo. Omnikv: Dynamic context selection for efficient long-context llms. In *International Conference on Learning Representations*, 2025.

[17] Yefei He, Luoming Zhang, Weijia Wu, Jing Liu, Hong Zhou, and Bohan Zhuang. Zipcache: Accurate and efficient kv cache quantization with salient token identification. In *Advances in Neural Information Processing Systems*, 2024.

[18] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization. In *Advances in Neural Information Processing Systems*, 2024.

[19] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.

[20] Jean Kaddour, Joshua Harris, Maximilian Mozes, Herbie Bradley, Roberta Raileanu, and Robert McHardy. Challenges and applications of large language models. *arXiv preprint arXiv:2307.10169*, 2023.

[21] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.

[22] Jiedong Lang, Zhehao Guo, and Shuyu Huang. A comprehensive study on quantization techniques for large language models. In *2024 4th International Conference on Artificial Intelligence, Robotics, and Communication*, 2024.

[23] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. Snapkv: Llm knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, 2024.

[24] Stephanie Lin, Jacob Hilton, and Owain Evans. Truthfulqa: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2022.

[25] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024.

[26] Stefano Markidis, Steven Wei Der Chien, Erwin Laure, Ivy Bo Peng, and Jeffrey S. Vetter. Nvidia tensor core programmability, performance & precision. In *2018 IEEE International Parallel and Distributed Processing Symposium Workshops*, 2018.

[27] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.

[28] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2024.

[29] Leonid Pekelis, Michael Feil, Forrest Moret, Mark Huang, and Tiffany Peng. Llama 3 gradient: A series of long context models, 2024. URL https://gradient.ai/blog/scaling-rotational-embeddings-for-long-context-language-models.

[30] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems*, 2023.

[31] Siva Reddy, Danqi Chen, and Christopher D. Manning. Coqa: A conversational question answering challenge. *Transactions of the Association for Computational Linguistics*, 2019.

[32] Bita Darvish Rouhani, Ritchie Zhao, Ankit More, Mathew Hall, Alireza Khodamoradi, Summer Deng, Dhruv Choudhary, Marius Cornea, Eric Dellinger, Kristof Denolf, et al. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537*, 2023.

[33] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: an adversarial winograd schema challenge at scale. *Communications of the ACM*, 2021.

[34] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[35] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 2024.

[36] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context llm inference. *arXiv preprint arXiv:2406.10774*, 2024.

[37] TII Team. Falcon 3 family of open foundation models, 2024. URL https://huggingface.co/tiiuae/Falcon3-7B-Base.

[38] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[39] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 2022.

[40] Ronald J. Williams and Jing Peng. An efficient gradient-based algorithm for on-line training of recurrent network trajectories. *Neural Computation*, 1990.

[41] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020.

[42] Tianyi Zhang, Jonah Yi, Zhaozhuo Xu, and Anshumali Shrivastava. Kv cache is 1 bit per channel: Efficient large language model inference with coupled quantization. In *Advances in Neural Information Processing Systems*, 2024.

[43] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang "Atlas" Wang, and Beidi Chen. H2o: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023.

[44] Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Huazuo Gao, Jiashi Li, Liyue Zhang, Panpan Huang, Shangyan Zhou, Shirong Ma, Wenfeng Liang, Ying He, Yuqing Wang, Yuxuan Liu, and Y.X. Wei. Insights into deepseek-v3: Scaling challenges and reflections on hardware for ai architectures. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025.

[45] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Zican Dong, Yupeng Hou, Beichen Zhang, Yingqian Min, Junjie Zhang, Peiyu Liu, Xiaolei Wang, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Yiwen Hu, Jian-Yun Nie, and Ji-Rong Wen. A survey of large language models. *Frontiers of Computer Science*, 2026.

[46] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. Sglang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, 2024.

Appendix

A Related Work

Quantization has emerged as one of the most hardware-friendly methods for KV cache compression, as it reduces the bit-width of cached key and value activations without altering the attention mechanism, remaining compatible with modern tensor-core hardware [26]. Existing methods can be broadly categorized by how they compute and store metadata (e.g., zero points, scaling factors, and outlier indices).

KIVI. KIVI [25] is a tuning-free, plug-and-play asymmetric quantization framework that applies per-channel quantization to key activations (to handle structured outlier channels) and per-token quantization to value activations, using static group-wise statistics computed over a fixed window of tokens (group size 32, 64, or 128). A residual cache stores recent tokens in FP16 until a threshold is reached, partially alleviating quantization error near the current token. KIVI achieves up to a $2.6\times$ memory reduction and $3.47\times$ throughput improvement, and has been adopted in Hugging Face `Transformers` [41]. However, from the perspective of ADKV, KIVI suffers from two intrinsic limitations. First, group-wise zero points and scaling factors must be stored as auxiliary metadata, accounting for 20% of the compressed KV cache. Second, the static fixed-window statistics cannot adapt to the sinusoidal-like temporal drift of post-RoPE outlier channels [35]: as the window size grows, the fixed-window statistics increasingly misrepresent the true activation distribution, leading to quantization error accumulation in long-context settings.

KVQuant. KVQuant [18] introduces a suite of techniques targeting sub-4-bit KV quantization at million-token context lengths. Its core innovations include: (1) pre-RoPE per-channel key quantization, which quantizes keys before the rotational transformation is applied to avoid post-RoPE distribution distortions; (2) non-uniform quantization (NUQ), which uses sensitivity-weighted, offline-calibrated datatypes per layer to better fit non-Gaussian activation distributions; and (3) per-vector dense-and-sparse quantization, which isolates the top-1% outlier values per vector and stores them separately in sparse format. KVQuant enables 1M context inference on a single A100-80GB GPU. Nevertheless, from the perspective of ADKV, KVQuant still requires storing per-group scaling factors, per-vector outlier indices, and offline-calibrated datatypes as auxiliary metadata, with total metadata comprising approximately 15.3% of the quantized KV cache. Furthermore, the dense-and-sparse outlier separation incurs additional memory management complexity.

MXFP4. MXFP4 [32] is an OCP-standardized 4-bit floating-point format designed for hardware-efficient inference and training. Each value is encoded in E2M1 format (1 sign bit, 2 exponent bits, 1 mantissa bit), and every block of 32 values shares a single 8-bit block-scale exponent (E8M0), providing a lightweight form of per-group dynamic range adaptation. From the perspective of ADKV, however, MXFP4 still embodies the static per-group quantization paradigm: the shared block scale is computed independently per block without exploiting cross-token temporal continuity. While the E8M0 scale is compact, it does not eliminate metadata storage: each block of 32 values still requires a shared 8-bit exponent.

ADKV addresses the two limitations shared by all of the above methods: (1) non-negligible metadata overhead, and (2) static statistics that cannot adapt to the temporal drift of KV cache distributions. Unlike KIVI and KVQuant, which compute statistics over fixed windows and store them explicitly, ADKV employs an EMA-based quantizer to recurrently track channel-wise zero points and scaling factors online, and embeds the associated metadata directly into the quantized sequence via adaptive delta coding, achieving an ultra-low memory footprint. Unlike the above methods, which rely on static statistics without exploiting the temporal structure of the KV cache, ADKV explicitly models and exploits this temporal structure as a first-class design principle. As a result, ADKV occupies a distinct position in the KV quantization landscape: a method to simultaneously achieve ultra-low metadata overhead and temporally adaptive quantization, reducing quantization metadata memory footprint by 99.76–99.94% over the static per-group quantization baselines while maintaining accuracy closely comparable to FP16 under long-context settings.

B Gradient Through the Attention Loss

Attention Output Gradient. Let $O = AV \in \mathbb{R}^{H \times T_{\mathrm{q}} \times d_h}$ be the reference attention output and $\hat{O} = \hat{A}\hat{V}$ the quantized output, where

$$A = \mathrm{Softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right), \quad \hat{A} = \mathrm{Softmax}\left(\frac{Q\hat{K}^\top}{\sqrt{d_h}}\right) \tag{11}$$

The per-sample loss is:

$$\mathcal{L}_{\text{attn}} = \frac{1}{H}\sum_{h=0}^{H-1}\frac{1}{|\Omega|}\|\hat{O} - O\|_F^2, \quad |\Omega| = T_{\mathrm{q}} \cdot d_h \tag{12}$$

The upstream gradient at the quantized output is:

$$\delta\hat{O}_{h,t,d} \triangleq \frac{\partial \mathcal{L}_{\text{attn}}}{\partial \hat{O}_{h,t,d}} = \frac{2}{|\Omega| \cdot H}\big(\hat{O} - O\big)_{h,t,d} \tag{13}$$

Gradient w.r.t. Quantized Value Cache $\hat{V}$. From $\hat{O} = \hat{A}\hat{V}$ (matrix product over the key/value dimension):

$$\delta\hat{V}_{h,t,d} \triangleq \frac{\partial \mathcal{L}_{\text{attn}}}{\partial \hat{V}_{h,t,d}} = \sum_{p=0}^{T_{\mathrm{q}}-1} \hat{A}_{h,p,t}\,\delta\hat{O}_{h,p,d} \tag{14}$$

Gradient w.r.t. Quantized Key Cache $\hat{K}$. The gradient at the attention weight matrix is:

$$\delta\hat{A}_{h,p,t} \triangleq \frac{\partial \mathcal{L}_{\text{attn}}}{\partial \hat{A}_{h,p,t}} = \sum_{d=0}^{d_h-1} \delta\hat{O}_{h,p,d}\,\hat{V}_{h,t,d} \tag{15}$$

The softmax backward (standard result for $\hat{A}_{h,p,\cdot} = \mathrm{Softmax}(\hat{S}_{h,p,\cdot})$) gives the gradient at the pre-softmax scores:

$$\delta\hat{S}_{h,p,t} \triangleq \frac{\partial \mathcal{L}_{\text{attn}}}{\partial \hat{S}_{h,p,t}} = \hat{A}_{h,p,t}\left[\delta\hat{A}_{h,p,t} - \sum_{l=0}^{T_{\text{kv}}-1} \hat{A}_{h,p,l}\,\delta\hat{A}_{h,p,l}\right] \tag{16}$$

The gradient at $\hat{K}$ follows from $\hat{S}_{h,p,t} = Q_{h,p,\cdot}\hat{K}^\top_{h,t,\cdot}/\sqrt{d_h}$:

$$\frac{\partial \mathcal{L}_{\text{attn}}}{\partial \hat{K}_{h,t,d}} = \frac{1}{\sqrt{d_h}}\sum_{p=0}^{T_{\mathrm{q}}-1} \delta\hat{S}_{h,p,t}\,Q_{h,p,d} \tag{17}$$

C Learned Step Size Quantization

For the quantization function in Equation (4), we employ a common method to estimate its gradient, termed STE:

$$\frac{\partial \hat{v}}{\partial v} = \begin{cases} 1, & \text{if } Q_{\mathrm{L}} \le \frac{v}{s} \le Q_{\mathrm{U}} \\ 0, & \text{otherwise} \end{cases} \tag{18}$$

where v is the original value and $\hat{v}$ is its quantized value. When the scaled value v/s is within the quantization range $[Q_{\mathrm{L}}, Q_{\mathrm{U}}]$, the gradient is estimated as 1. Otherwise, the gradient is blocked. On the other hand, for the scaling factor s, we employ LSQ to estimate its gradient:

$$\mathrm{LSQ}(v, s, \hat{v}) = \frac{\partial \hat{v}}{\partial s} = \begin{cases} -\frac{v}{s} + \left\lfloor \frac{v}{s} \right\rceil, & \text{if } Q_{\mathrm{L}} \leq \frac{v}{s} \leq Q_{\mathrm{U}} \\ Q_{\mathrm{L}}, & \text{if } \frac{v}{s} < Q_{\mathrm{L}} \\ Q_{\mathrm{U}}, & \text{if } \frac{v}{s} > Q_{\mathrm{U}} \end{cases} \tag{19}$$

This gradient is derived by using the STE to approximate the gradient of the quantization function in Equation (4).

D Gradient Through the ADKV Quantizer

For clarity, for a given head h and hidden dimension d, we define $\delta\hat{x}_t$ as the upstream gradient, i.e., $\delta\hat{x}_t = \partial\mathcal{L}_{\text{attn}}/\partial\hat{V}_{h,t,d}$ or $\delta\hat{x}_t = \partial\mathcal{L}_{\text{attn}}/\partial\hat{K}_{h,t,d}$, as derived in Appendix B. Gradients at node $\widehat{Resi}_t$ are accumulated from all its successors:

$$\delta\widehat{Resi}_t \triangleq \frac{\partial\mathcal{L}_{\text{attn}}}{\partial\widehat{Resi}_t} = \underbrace{\delta\hat{x}_t}_{\text{from } \hat{x}_t} + \underbrace{\alpha\,\delta ZP_{t+1}}_{\text{from } ZP_{t+1}} \tag{20}$$

The STE backward then gives $\delta Resi_t \triangleq \partial\mathcal{L}_{\text{attn}}/\partial Resi_t = \delta\widehat{Resi}_t \cdot \mathbf{1}_t$ according to Equation (18), where $\mathbf{1}_t = \mathbf{1}[Q_{\mathrm{L}} \leq Resi_t/S_t \leq Q_{\mathrm{U}}]$. For clipped inputs ($\mathbf{1}_t = 0$), the gradient is zero. Gradients at node ZP_t are accumulated from all its successors:

$$\delta ZP_t \triangleq \frac{\partial\mathcal{L}_{\text{attn}}}{\partial ZP_t} = \underbrace{\delta\hat{x}_t}_{\text{from } \hat{x}_t} + \underbrace{(-1)\cdot\delta\widehat{Resi}_t \cdot \mathbf{1}_t}_{\text{from } Resi_t} + \underbrace{\delta ZP_{t+1}}_{\text{from } ZP_{t+1}} \tag{21}$$

Substituting Equation (20) into Equation (21), we obtain:

$$\delta ZP_t = \delta\hat{x}_t\big(1 - \mathbf{1}_t\big) + \delta ZP_{t+1}\big(1 - \alpha\mathbf{1}_t\big) \tag{22}$$

This backward recurrence runs from $t = k-1$ down to $t = 0$ with boundary condition $\delta ZP_k = 0$. The boundary condition is enforced by the truncated backpropagation through time (TBPTT) [40] operation at each chunk boundary.

The scaling factor update in Equation (5) gives the local Jacobian:

$$\frac{\partial S_{t+1}}{\partial S_t} = \mathcal{A}_t\big(1 + \beta e_t\big) \tag{23}$$

where $\mathcal{A}_t = \mathbf{1}[S_{\text{min}} \leq S_t(1+\beta e_t) \leq S_{\text{max}}]$ is 0 when the clamp is active (gradient blocked) and 1 otherwise. The backward signal at S_t collects the LSQ gradient in Equation (19) from the current step and the propagated signal from the future:

$$\delta S_t \triangleq \frac{\partial\mathcal{L}_{\text{attn}}}{\partial S_t} = \frac{\delta\widehat{Resi}_t}{Q_{\mathrm{U}}} \cdot \mathrm{LSQ}(Resi_t, S_t, \widehat{Resi}_t) + \delta S_{t+1} \cdot \mathcal{A}_t \cdot \big(1 + \beta e_t\big) \tag{24}$$

with $\delta S_k = 0$ (detached chunk boundary).

Gradient w.r.t. Initial Zero Point ZP_0. The initial state ZP_0 feeds into the zero point recurrence of chunk 0 only (all subsequent chunks detach the state). Therefore:

$$\frac{\partial\mathcal{L}_{\text{attn}}}{\partial ZP_0} = \delta ZP_0^0 \tag{25}$$

where δZP_0^0 is the backward signal at the start of chunk 0, obtained by running the recurrence in Equation (22) from $t = k-1$ down to $t = 0$ within chunk 0.

Gradient w.r.t. Initial Scaling Factor S_0. The initial scaling factor S_0 feeds into chunk 0 only. Therefore:

$$\frac{\partial \mathcal{L}_{\text{attn}}}{\partial S_0} = \delta S_0^0 \tag{26}$$

where δS_0^0 is the backward signal at the start of chunk 0, obtained by running the recurrence in Equation (24) from $t = k-1$ down to $t = 0$ within chunk 0.

Gradient w.r.t. Zero Point Tracking Rate α. The parameter α participates in every chunk via the update in Equation (5) at each step t. Differentiating with respect to α (treating $\widehat{Resi}_t$ as a node in the autograd graph), we obtain:

$$\frac{\partial \mathcal{L}_{\text{attn}}}{\partial \alpha} = \sum_{m=0}^{n-1} \sum_{t=0}^{k-1} \delta ZP_{t+1}^m \, \widehat{Resi}_t^m \tag{27}$$

Gradient w.r.t. Scaling Factor Tracking Rate β. The parameter β participates in every chunk via the update in Equation (5). Differentiating with respect to β, we obtain:

$$\frac{\partial \mathcal{L}_{\text{attn}}}{\partial \beta} = \sum_{m=0}^{n-1} \sum_{t=0}^{k-1} \delta S_{t+1}^m \cdot \mathcal{A}_t^m \cdot S_t^m \cdot e_t^m \tag{28}$$

Gradient w.r.t. Quantization Target T. The parameter T enters the error as $e_t = |Q_t| - T$, so $\partial e_t / \partial T = -1$. The per-step gradient contribution through Equation (5) is $\delta S_{t+1} \cdot \mathcal{A}_t \cdot S_t \cdot \beta \cdot (-1)$:

$$\frac{\partial \mathcal{L}_{\text{attn}}}{\partial T} = -\beta \sum_{m=0}^{n-1} \sum_{t=0}^{k-1} \delta S_{t+1}^m \cdot \mathcal{A}_t^m \cdot S_t^m \tag{29}$$

E ADKV System Analysis

In this section, we present a rigorous theoretical analysis of the ADKV system, an adaptive delta quantization method for KV cache, which combines per-channel zero point offset tracking with multiplicative automatic gain control (AGC).

E.1 Notation and Signal Model

Let $x_t \in \mathbb{R}^{H \times d_h}$ denote the key or value vector at decoding step t. ADKV processes each channel (h, d) independently. For clarity, we adopt the signal analysis notation and omit the channel subscript. The ADKV quantizer maintains the following per-channel state variables (Table 3):

Table 3: ADKV System Symbols

Symbol	Parameter	Semantics
μ_t	$ZP_t^{h,d}$	Running DC-offset estimate
σ_t	$S_t^{h,d}$	Quantization step size
α	$\alpha^{h,d}$	DC update coefficient
β	$\beta^{h,d}$	Step-size update coefficient
q^*	$T^{h,d}$	Target quantization magnitude

For reference, we recall the ADKV system equations:

$$r_t = x_t - \mu_t \qquad \text{(residual after DC removal)} \tag{30}$$

$$q_t = \mathrm{Clamp}\left(\left\lfloor \frac{r_t}{\sigma_t} \right\rceil, Q_\mathrm{L}, Q_\mathrm{U}\right) \qquad \text{(residual quantization)} \tag{31}$$

$$\hat{r}_t = q_t \cdot \sigma_t, \quad \hat{x}_t = \hat{r}_t + \mu_t \qquad \text{(reconstruction)} \tag{32}$$

$$\mu_{t+1} = \mu_t + \alpha\, \hat{r}_t \qquad \text{(DC update)} \tag{33}$$

$$\sigma_{t+1} = \mathrm{Clamp}\left(\sigma_t(1 + \beta(|q_t| - q^*)),\, \sigma_{\min},\, \sigma_{\max}\right) \qquad \text{(step-size update)} \tag{34}$$

The per-step reconstruction error is $e_t = x_t - \hat{x}_t = r_t - q_t \cdot \sigma_t$. This error decomposes into three regimes depending on whether the residual falls within the representable range:

$$e_t = \begin{cases} r_t - \sigma_t \left\lfloor r_t/\sigma_t \right\rceil, & \text{if } Q_\mathrm{L}\sigma_t \le r_t \le Q_\mathrm{U}\sigma_t \quad \text{(granular region)} \\ r_t - Q_\mathrm{U}\sigma_t, & \text{if } r_t > Q_\mathrm{U}\sigma_t \quad \text{(upper clipping region)} \\ r_t - Q_\mathrm{L}\sigma_t, & \text{if } r_t < Q_\mathrm{L}\sigma_t \quad \text{(lower clipping region)} \end{cases} \tag{35}$$

Under the high-resolution approximation [4], the quantization error in the granular region is modeled as $e_t \sim \mathcal{U}(-\sigma_t/2, \sigma_t/2)$, and the residual is assumed to follow $r_t \sim \mathcal{N}(0, \sigma_r^2)$. Under these assumptions, the total MSE is given by:

$$\begin{aligned} D_\text{total} &= D_\text{gran} + D_\text{clip}^{(\mathrm{U})} + D_\text{clip}^{(\mathrm{L})} \\ &= \frac{\sigma_t^2}{12}\left[\Phi(\lambda_\mathrm{U}) - \Phi(\lambda_\mathrm{L})\right] \\ &\quad + \sigma_r^2\left[Q(\lambda_\mathrm{U})(1+\lambda_\mathrm{U}^2) - \lambda_\mathrm{U}\phi(\lambda_\mathrm{U})\right] \\ &\quad + \sigma_r^2\left[Q(-\lambda_\mathrm{L})(1+\lambda_\mathrm{L}^2) + \lambda_\mathrm{L}\phi(\lambda_\mathrm{L})\right] \end{aligned} \tag{36}$$

where $\phi(x) = 1/\sqrt{2\pi}e^{-x^2/2}$, $\Phi(x) = \int_{-\infty}^{x} 1/\sqrt{2\pi}e^{-u^2/2}du$, and $Q(x) = \int_{x}^{\infty} 1/\sqrt{2\pi}e^{-u^2/2}du$ are the probability density function (PDF), cumulative distribution function (CDF), and complementary cumulative distribution function (CCDF) of the standard normal distribution, respectively. The **overload ratios** are defined as $\lambda_\mathrm{U} = Q_\mathrm{U}\sigma_t/\sigma_r$ and $\lambda_\mathrm{L} = Q_\mathrm{L}\sigma_t/\sigma_r$.

In the symmetric case where $Q_\mathrm{L} = -Q_\mathrm{U}$, the overload ratio reduces to $\lambda = Q_\mathrm{U}\sigma_t/\sigma_r$:

$$\begin{aligned} D_\text{total} &= D_\text{gran} + D_\text{clip} \\ &= \frac{\sigma_t^2}{12}\left[1 - 2Q(\lambda)\right] + 2\sigma_r^2\left[(1+\lambda^2)Q(\lambda) - \lambda\phi(\lambda)\right] \end{aligned} \tag{37}$$

E.2 DC-Tracker Analysis

Substituting $\hat{r}_t = (x_t - \mu_t) - e_t$ into the DC update in Equation (33) yields:

$$\mu_{t+1} = (1-\alpha)\mu_t + \alpha x_t - \alpha e_t \tag{38}$$

This is a first-order IIR low-pass filter applied to x_t, with time constant $\tau_\mathrm{DC} = 1/\alpha$ steps, driven by the quantization error e_t as additive noise. For a stationary process with $\mathbb{E}[x_t] = \bar{x}$ and $\mathbb{E}[e_t] = 0$ (zero-mean quantization noise), imposing $\mathbb{E}[\mu_{t+1}] = \mathbb{E}[\mu]$ at stationarity gives $\mathbb{E}[\mu_\infty] = \bar{x}$. The DC tracker is therefore **asymptotically unbiased**. Define the tracking error as $\varepsilon_t = \bar{x} - \mu_t$. For a constant-mean signal, its dynamics follow $\varepsilon_{t+1} = (1-\alpha)\varepsilon_t + \alpha e_t$. Assuming stationarity $V_\varepsilon \triangleq \mathrm{Var}[\varepsilon_{t+1}] = \mathrm{Var}[\varepsilon_t]$, and invoking the independence between ε_t and e_t (under a zero-delay feedback approximation), we obtain:

$$V_\varepsilon = \frac{\alpha^2}{1-(1-\alpha)^2}\, D_\mathrm{total} = \frac{\alpha}{2-\alpha}\, D_\mathrm{total} \approx \frac{\alpha}{2}\, D_\mathrm{total} \quad (\alpha \ll 1) \tag{39}$$

The Z-domain transfer function from quantization error e_t to DC tracking error ε_t is:

$$H_\varepsilon(z) = \frac{\alpha z^{-1}}{1 - (1-\alpha)z^{-1}} \tag{40}$$

The resulting power spectral density is:

$$S_\varepsilon(f) = \frac{\alpha^2 D_{\text{total}}}{\left|1 - (1-\alpha)e^{-j2\pi f}\right|^2} \tag{41}$$

This is **low-frequency noise shaping**: quantization error energy is concentrated at DC by the tracking loop, implying that slowly varying signals accumulate correlated drift.

For a linearly drifting signal $x_t = \bar{x} + vt$, the steady-state lag of the DC tracker is $\text{Lag}_{\text{DC}} = v/\alpha$ per step. Following a step change of magnitude Δ, the tracking error decays exponentially with time constant $\tau_{\text{DC}} = 1/\alpha$ steps.

E.3 Step-Size AGC Analysis

Ignoring clipping boundary effects, the stationarity condition $\mathbb{E}[\sigma_{t+1}] = \mathbb{E}[\sigma_t]$ applied to the multiplicative update in Equation (34) yields:

$$\mathbb{E}[|q_t|] = q^* \tag{42}$$

The AGC thus **locks the mean absolute quantization index to the learned target** q^*. For $r_t \sim \mathcal{N}(0, \sigma_r^2)$, neglecting clipping: $\mathbb{E}[|q_t|] \approx \mathbb{E}\left[|r_t/\sigma_t|\right] = \sqrt{2/\pi} \cdot \sigma_r/\sigma_t$. Consequently, the steady-state step size satisfies:

$$\sigma^* = \sqrt{2/\pi} \cdot \sigma_r / q^* \tag{43}$$

Let $u_t = \log(\sigma_t/\sigma^*)$ denote the logarithmic deviation of the step size from its steady-state value. Linearizing the AGC update around σ^* yields a first-order approximation of the dynamics. Defining the sensitivity coefficient:

$$\gamma = -\frac{d\,\mathbb{E}[|q_t|]}{d\log\sigma}\bigg|_{\sigma^*} = q^* \tag{44}$$

the evolution of the mean deviation can be expressed as $\mathbb{E}[u_{t+1}] \approx (1 - \beta q^*)\,\mathbb{E}[u_t]$. Consequently, **the system is stable if and only if** $0 < \beta < 2/q^*$, and the corresponding convergence time constant is given by $\tau_\sigma = 1/\beta q^*$ steps.

The steady-state variance of u_t satisfies:

$$\text{Var}[u_\infty] \approx \frac{\beta}{2q^*}\text{Var}[|q_t|] \tag{45}$$

The additional granular noise introduced by step-size fluctuation around σ^* is:

$$\Delta D_{\text{jitter}} \approx D^*_{\text{gran}} \cdot \frac{\beta \text{Var}[|q_t|]}{q^*} \tag{46}$$

Optimal Quantization Target T^*. For the Gaussian case, substituting σ^* from Equation (43) into Equation (37) yields the total MSE as a function of the overload ratio $\lambda = Q_{\text{U}}\sqrt{2/\pi}/q^*$ (assuming $Q_{\text{L}} = -Q_{\text{U}}$):

$$D^*_{\text{total}}(\lambda) = \frac{\sigma_r^2}{6\pi q^{*2}}\big[1 - 2Q(\lambda)\big] + D_{\text{clip}}(\lambda) \tag{47}$$

Setting $dD^*_{\text{total}}/d\lambda = 0$ yields the implicit optimality condition:

$$4[\phi(\lambda) - \lambda Q(\lambda)] = \frac{\lambda(1 - 2Q(\lambda)) + \lambda^2\phi(\lambda)}{6Q_{\text{U}}^2} \tag{48}$$

Numerically, the 4-bit Gaussian-optimal solution is $\lambda^* \approx 2.5$, giving:

$$q^{*\,\text{opt}} = \frac{Q_{\text{U}}\sqrt{2/\pi}}{\lambda^*} = \frac{7 \times 0.798}{2.5} \approx 2.23 \tag{49}$$

Coupled Self-Consistent Equations at Stationarity. To summarize the above analysis, the ADKV system leads to the following **self-consistent system**:

$$\begin{cases} \sigma^* = \sqrt{\frac{2}{\pi}} \cdot \frac{\sigma_r}{q^*} \\ D^* = \frac{\sigma^{*2}}{12} + D_{\text{clip}}(\sigma^*) \\ \sigma_r^2 = \sigma_x^2 + \dfrac{\alpha}{2-\alpha} D^* \end{cases} \tag{50}$$

where $\sigma_x^2 = \text{Var}[x_t]$ is the true signal variance.

E.4 Composite Error Budget

Aggregating all sources, the total reconstruction MSE at stationarity is:

$$D_{\text{rec}} \approx \underbrace{\frac{\sigma_t^2}{12}}_{\text{granular}} + \underbrace{\frac{\alpha}{2-\alpha} \cdot \frac{\sigma_t^2}{12}}_{\text{DC tracker}} + \underbrace{\frac{\beta\,\text{Var}[|q_t|]}{q^*} \cdot \frac{\sigma_t^2}{12}}_{\text{step-size jitter}} + \underbrace{D_{\text{clip}}(\lambda)}_{\text{clipping}} \tag{51}$$

ADKV adopts an SGD-based calibration framework to obtain a set of optimal parameters that minimize this objective.

E.5 Error Propagation to Attention Output

Let T_{kv} denote the length of the KV cache sequence. Key quantization error e_j^{k} at position j perturbs the attention logit between query i and key j:

$$\Delta s_{ij} = \frac{q_i^\top e_j^{\text{k}}}{\sqrt{d_h}}, \quad \text{Var}[\Delta s_{ij}] \approx \frac{D_{\text{k}}}{d_h}\|q_i\|^2 \tag{52}$$

The attention output perturbation due to value quantization errors is:

$$\Delta o_i = \sum_{j=0}^{T_{\text{kv}}-1} a_{ij} e_j^{\text{v}} \tag{53}$$

where $\{a_{ij}\}$ are the softmax attention weights. Assuming approximately uniform weights $a_{ij} \approx 1/T_{\text{kv}}$ and independence of e_j^{v}, we obtain:

$$\text{Var}[\Delta o_i] \approx \frac{1}{T_{\text{kv}}} D_{\text{v}}. \tag{54}$$

E.6 Experiments on ADKV Stability

We analyze the token-wise MSE behavior of ADKV under 4/2-bit quantization over a 256K-token context length. The results report the relative MSE increase of each 10%-token window with respect to the initial $[0, 10\%]$ window. Specifically, ADKV denotes the original 4/2-bit quantization scheme, while ADKV-O represents an oracle variant using lossless FP16 recurrent feedback. The Diff metric is computed as $(\mathrm{MSE}_{\text{ADKV}} - \mathrm{MSE}_{\text{ADKV-O}})/\mathrm{MSE}_{\text{ADKV}}$, which isolates the additional error introduced by feeding 4/2-bit quantization results back into the EMA state update.

As shown in Figure 5, for both K and V states, ADKV and ADKV-O maintain nearly flat MSE trends across the entire 256K context, indicating that quantization error does not accumulate with increasing token positions. Although 2-bit feedback introduces a fixed quantization error bias compared with FP16 feedback, this error remains stable over long contexts rather than progressively increasing.

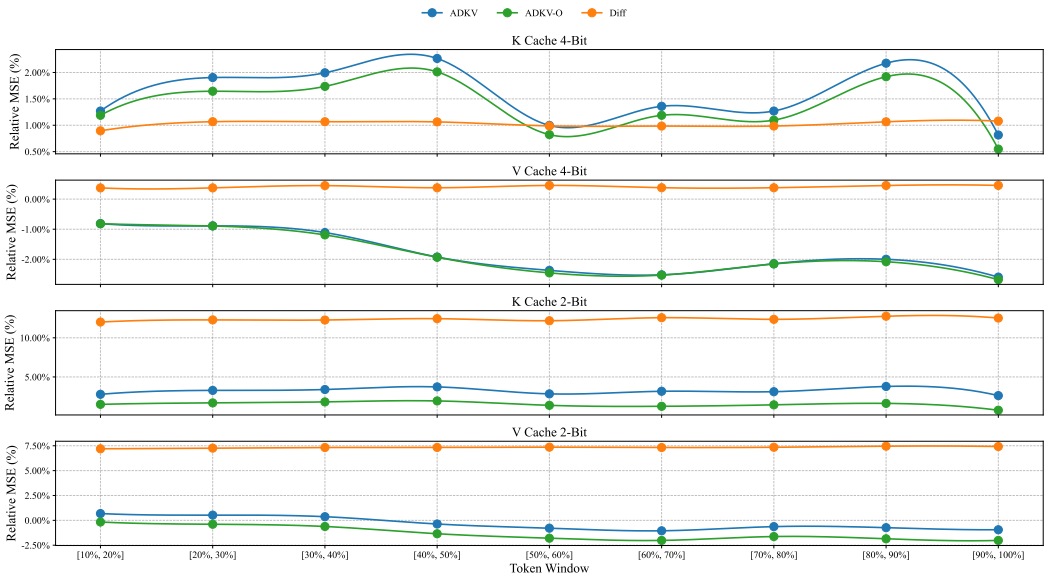


Figure 5: Relative MSE trends of ADKV and its oracle variant ADKV-O under 4/2-bit quantization over a 256K-token context length.

F Calibration Details

In this section, we provide a detailed technical description of the calibration framework for ADKV. As discussed in Appendix E, the calibration algorithm implements end-to-end SGD-based optimization of the five channel-wise quantizer parameters: initial zero point, initial scaling factor, zero point tracking rate, scaling factor tracking rate, and quantization target, via a combination of the STE, LSQ-style scaled gradients, constrained reparameterization, and TBPTT.

The ADKV quantizer during inference is a **stateful, causal, per-channel adaptive filter** whose behavior is entirely determined by five learned parameter vectors of length $H \times d_h$ (the number of KV channels). The training problem is therefore:

> Given a calibration set of key/value activation sequences $\{X^{(i)}\}$, find the parameter set $\Theta = \{ZP_0, S_0, \alpha, \beta, T\}$ that minimizes the expected reconstruction distortion under adaptive quantization dynamics.

Because the quantization operation (rounding) is non-differentiable, the training graph requires careful gradient approximation. The implementation addresses this with three complementary techniques described in detail below.

F.1 Differentiable Quantization: `IntQuantSTE`

`IntQuantSTE` is a `torch.autograd.Function` that implements the standard uniform scalar quantizer with clipping in Equation (4), returning both the reconstructed residual $\widehat{Resi}_t$ and the integer index Q_t. A numerical guard $S_t \leftarrow \max(S_t, \varepsilon)$ with $\varepsilon = 10^{-8}$ prevents division by zero.

For the backward pass, the standard STE passes the upstream gradient through the rounding operation for inputs within the representable range, and zeros it out in the clipping region as in Equation (18). The gradient of the scaling factor follows the LSQ formulation in Equation (19). The normalization factor $1/Q_{\mathrm{U}}$ stabilizes the gradient magnitude across different bit-widths, following the LSQ prescription. The implementation returns `grad_x_hat * grad_scale * lsq_norm` as the gradient for S_t, where `lsq_norm = 1.0 / upper_limit`.

F.2 Constrained Parameter Reparameterization

All five learnable parameters require range constraints to ensure that the stability conditions derived in Appendix E are respected throughout calibration. Rather than imposing hard constraints via projected gradient descent, the implementation uses smooth and unconstrained reparameterizations:

Initial Zero Point ZP_0.

$$ZP_0 = \theta_{\mu_0} \quad \text{(unconstrained)} \tag{55}$$

No constraint is required.

Initial Scaling Factor S_0.

$$S_0 = \exp(\theta_{\log \sigma_0}), \quad \theta_{\log \sigma_0} \in \mathbb{R} \tag{56}$$

where this reparameterization ensures that $S_0 > 0$ unconditionally. The exponential reparameterization also means the optimizer works on a log scale, which is the natural scale for multiplicative quantities and prevents the optimizer from proposing negative step sizes.

Zero Point Tracking Rate α.

$$\alpha = \mathrm{sigmoid}(\theta_\alpha) \in (0, 1) \tag{57}$$

The constraint $\alpha \in (0, 1)$ is necessary and sufficient for the DC-tracker IIR filter to be BIBO-stable (Appendix E). The sigmoid squashing ensures this holds for any value of the raw parameter θ_α.

Scaling Factor Tracking Rate β.

$$\beta = 0.1 \cdot \mathrm{sigmoid}(\theta_\beta) \in (0, 0.1) \tag{58}$$

The constraint $\beta > 0$ prevents ADKV from entering an unstable regime. The bound $|\beta| < 0.1$ combined with the stability condition $\beta < 2/q^*$ (with $q^{*\,\mathrm{opt}} \approx 2.23$) gives $0.1 < 0.9$, so the reparameterization is tighter than the theoretical stability margin (Appendix E) and it is sufficient for stability.

Quantization Target T.

$$T = \theta_{q^*} \quad \text{(unconstrained)} \tag{59}$$

While T is theoretically constrained to be positive (it represents a magnitude target), the optimization landscape in practice keeps it positive.

The `reset_parameters` method sets the following initial values (Table 4). The small random initialization introduces diversity across channels, breaking the symmetry that would otherwise prevent the optimizer from differentiating channel-specific adaptive behaviors.

Table 4: Initial Values of Parameters

Parameter	Initial Value	Rationale
`init_zp` (θ_{μ_0})	0.0	Zero point assumption
`init_log_scale` ($\theta_{\log \sigma_0}$)	0.0	$S_0 = \exp(0) = 1$
`alpha` (θ_α)	$\mathcal{N}(0, 0.02^2)$	$\alpha \approx \text{sigmoid}(0) = 0.5$
`beta` (θ_β)	$\mathcal{N}(0, 0.02^2)$	Small initial AGC gain
`target` (θ_{q^*})	$Q_{\text{U}}/2$	Mid-range magnitude target

F.3 Truncated Backpropagation Through Time (TBPTT)

The ADKV quantizer is a recurrent system whose states (ZP_t, S_t) at time t depend on all previous tokens. Full backpropagation through the entire sequence of length T would require $\mathcal{O}(T)$ memory and produce gradients with very long dependency chains, making optimization difficult due to vanishing or exploding gradients.

The input sequence is divided into non-overlapping chunks of length `truncation` $= k$:

$$x_{0:T-1} \to \left(x_{0:k-1}, x_{k:2k-1}, \ldots, x_{(n-1)k:nk-1}\right) \tag{60}$$

States are **propagated forward** across chunk boundaries (preserving causal correctness), but **gradients are detached** at each boundary. This is the standard TBPTT algorithm. Each chunk contributes an independent gradient computation of length exactly `truncation`, so:

- **Memory:** The memory complexity is $O(k \cdot H \cdot d_h)$ per chunk, instead of $O(T \cdot H \cdot d_h)$ for the full sequence.
- **Gradient Horizon:** Parameters that affect a chunk boundary do not receive gradients from future chunks, but the forward computation remains exact (no truncation of the forward pass).

The inner loop `_adkv_chunk_forward` is decorated with `@torch.compile`, triggering TorchInductor fusion of all elementwise operations within the loop, eliminating kernel launch overhead and intermediate tensor allocations.

F.4 Algorithm

We present the full ADKV calibration procedure in Algorithm 1, which details the step-by-step workflow for estimating and optimizing the quantizer parameters based on the calibration set.

F.5 ADKV Parameter Distribution

As shown in Figure 6, we provide the distribution of ADKV parameters on Llama-2 7B calibrated over the Wikipedia dataset. According to the statistical results, we provide the following insights:

- **Initial Zero Point ZP_0.** The initial zero points are predominantly centered around zero, indicating that, for most channels, the first few values begin near zero. However, we observe that a subset of channels exhibits relatively large initial zero points. These large initial zero points can facilitate faster convergence of the corresponding ADKV quantizer.
- **Initial Scaling Factor S_0.** The initial scaling factors reflect the average magnitude for the first few key/value activations. The K cache scaling factors are systematically larger than the V cache scaling factors.
- **Zero Point Tracking Rate α.** The zero point tracking rates reflect how aggressively the ADKV quantizer incorporates the incoming residual into the current zero point estimate. Empirically, the tracking rates for the K cache are consistently larger than those for the V cache, indicating that key activations are more readily tracked by the ADKV quantizer than value activations. This behavior can be attributed to RoPE, which applies a rotational transformation to key activations, resulting in the sinusoidal-like temporal drift. While such

Algorithm 1: ADKV Calibration for a Single Transformer Layer

Input: Calibration set $\mathcal{D} = \{(Q, K, V, M)\}_{i=0}^{N-1}$;
hyper-parameters: epochs E, batch size B, learning rate η, gradient clip τ, chunk length k, bits b, log-scale bounds $[\sigma_{\min}, \sigma_{\max}]$
Output: Learned K-quantizer parameters Θ_{K}, V-quantizer parameters Θ_{V}

// Derive integer range and scale bounds from bit-width

$Q_{\mathrm{U}} \leftarrow 2^{b-1} - 1$; $Q_{\mathrm{L}} \leftarrow -2^{b-1}$; $S_{\min} \leftarrow e^{\sigma_{\min}}$; $S_{\max} \leftarrow e^{\sigma_{\max}}$

// Initialize per-channel parameters for K and V independently

for $\Theta \in \{\Theta_K, \Theta_V\}$ **do**

  $\theta_{\mu_0} \leftarrow \mathbf{0}$; $\theta_{\log \sigma_0} \leftarrow \mathbf{0}$; $\theta_\alpha \sim \mathcal{N}(0, 0.02^2)$; $\theta_\beta \sim \mathcal{N}(0, 0.02^2)$; $\theta_{q^*} \leftarrow Q_{\mathrm{U}}/2$

$\mathcal{O}_{\mathrm{K}} \leftarrow \mathrm{Adam}(\Theta_{\mathrm{K}}, \eta)$; $\mathcal{O}_{\mathrm{V}} \leftarrow \mathrm{Adam}(\Theta_{\mathrm{V}}, \eta)$

for *epoch* $e \leftarrow 0$ **to** $E - 1$ **do**

  for *each mini-batch* $\{(Q, K, V, M)\}$ *of size* B *from* $\mathcal{D}$ **do**

    // — Forward pass —

    // Reparameterize raw parameters into constrained effective parameters

    $ZP_0 \leftarrow \theta_{\mu_0}$; $S_0 \leftarrow \exp(\theta_{\log \sigma_0})$; $\alpha \leftarrow \mathrm{sigmoid}(\theta_\alpha)$; $\beta \leftarrow 0.1 \cdot \mathrm{sigmoid}(\theta_\beta)$; $T \leftarrow \theta_{q^*}$

    for $i \leftarrow 0$ **to** $B - 1$ **do**

      // Compute reference attention output (no gradient)

      $O \leftarrow \mathrm{Softmax}\big(\frac{QK^\top}{\sqrt{d_h}} + M\big)\, V$

      // Reconstruct K and V via ADKV quantizer (differentiable)

      for $X \in \{K, V\}$ with corresponding $(ZP_0, S_0, \alpha, \beta, T)$ **do**

        // Pad to multiple of k; initialize recurrent states

        $ZP \leftarrow ZP_0$; $S \leftarrow S_0$

        for *chunk* $m \leftarrow 0$ **to** $\lceil T/k \rceil - 1$ **do**

          for *step* $t \leftarrow 0$ **to** $k - 1$ **do**

            // Zero point removal

            $Resi \leftarrow X[mk{+}t] - ZP$

            // Quantize & dequantize (STE+LSQ in backward)

            $Q \leftarrow \mathrm{Clamp}(\lfloor Resi/S \rceil,\, Q_{\mathrm{L}},\, Q_{\mathrm{U}})$; $\widehat{Resi} \leftarrow Q \cdot S$

            $\hat{X}[mk{+}t] \leftarrow \widehat{Resi} + ZP$

            // Zero point update

            $ZP \leftarrow ZP + \alpha \cdot \widehat{Resi}$

            // Scaling factor update

            $S \leftarrow \mathrm{Clamp}\big(S \cdot (1 + \beta(|Q| - T)),\, S_{\min},\, S_{\max}\big)$

          // TBPTT: cut gradient across chunk boundary

          $ZP \leftarrow ZP$`.detach()`; $S \leftarrow S$`.detach()`

      // Compute quantized attention output

      $\hat{O} \leftarrow \mathrm{Softmax}\big(\frac{Q\hat{K}^\top}{\sqrt{d_h}} + M\big)\, \hat{V}$

      // Attention-output distortion

      $\mathcal{L}_{\text{attn}} \mathrel{+}= \big\|\hat{O} - O\big\|_F^2 / |\Omega|$

    $\mathcal{L}_{\text{attn}} \leftarrow \mathcal{L}_{\text{attn}}/B$

    // — Backward pass and parameter update —

    $\mathcal{O}_{\mathrm{K}}$`.zero_grad()`; $\mathcal{O}_{\mathrm{V}}$`.zero_grad()`

    $\mathcal{L}_{\text{attn}}$`.backward()`;

    `clip_grad_norm`$(\Theta_{\mathrm{K}}, \tau)$; `clip_grad_norm`$(\Theta_{\mathrm{V}}, \tau)$

    $\mathcal{O}_{\mathrm{K}}$`.step()`; $\mathcal{O}_{\mathrm{V}}$`.step()`

return $\Theta_{\mathrm{K}},\ \Theta_{\mathrm{V}}$

structured variations can degrade the effectiveness of static per-group quantization, ADKV can effectively compensate for them through adaptive zero point updates.

- **Scaling Factor Tracking Rate β.** The scaling factor tracking rates must remain sufficiently small to ensure the stability of the ADKV system, as discussed in Appendix E. Notably, the V cache exhibits relatively larger scaling factor tracking rates at the first few layers, indicating greater variability in the residuals at these early layers.
- **Quantization Target T.** The initial value of T is $Q_{\mathrm{U}}/2 = 3.5$, but the learned distribution shifts toward lower values concentrated around 2.0–2.5, closely bracketing the Gaussian-optimal prediction $q^*_{\mathrm{opt}} \approx 2.23$ derived in Equation (49).

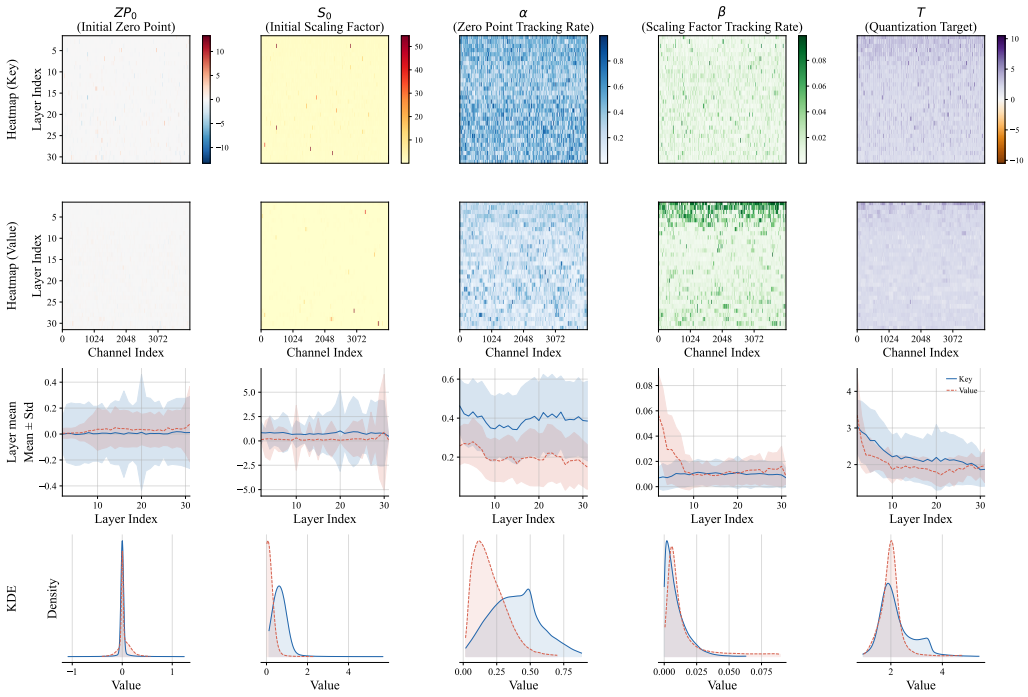


Figure 6: ADKV parameter distributions on Llama-2 7B calibrated over the Wikipedia dataset. The figure presents layer-wise statistics and global distributions of ADKV parameters. Specifically, the first two rows visualize heatmaps of ADKV parameters across layers and channels, highlighting their magnitude patterns and structural variations. The third row reports the layer-wise mean and standard deviation, providing a concise summary of distributional trends along layer depth. The final row shows kernel density estimation (KDE) curves, revealing the overall value distributions.

G Memory Analysis

In this section, we present a comprehensive memory analysis of ADKV. The ADKV quantizer involves seven parameters in total. Table 5 summarizes the formalized memory footprint. Notably, the state variables $ZP_t^{h,d}$ and $S_t^{h,d}$ are computed online and maintained in registers during execution, thereby incurring no persistent memory overhead.

This design implies that ADKV only requires a negligible amount of memory to store channel-wise metadata ($\alpha^{h,d}$, $\beta^{h,d}$, $T^{h,d}$, $ZP_0^{h,d}$ and $S_0^{h,d}$), which is independent of context length. Consequently, the overall memory footprint of ADKV does not scale with sequence length, in contrast to static per-group quantization whose cost grows linearly with T. From a systems perspective, this decoupling effectively shifts the memory bottleneck away from metadata, enabling more efficient utilization of limited GPU memory resources and allowing longer contexts to be accommodated under the same memory budget.

Table 5: Memory Footprint of ADKV Quantizer Parameters. Let L, H, d_h, and b_p denote the number of layers, attention heads, hidden dimension, and bytes required to store each parameter (e.g., 2 for FP16, 4 for FP32), respectively. Let T and b_t denote the context length and the number of bytes required to store the compressed KV cache, respectively.

Parameter	Memory Footprint	Proportion
$ZP_t^{h,d}$	Stored in registers online	-
$S_t^{h,d}$	Stored in registers online	-
$\alpha^{h,d}$	$L \times H \times d_h \times 2 \times b_p$	$\frac{b_p}{T \times b_t}$
$\beta^{h,d}$	$L \times H \times d_h \times 2 \times b_p$	$\frac{b_p}{T \times b_t}$
$T^{h,d}$	$L \times H \times d_h \times 2 \times b_p$	$\frac{b_p}{T \times b_t}$
$ZP_0^{h,d}$	$L \times H \times d_h \times 2 \times b_p$	$\frac{b_p}{T \times b_t}$
$S_0^{h,d}$	$L \times H \times d_h \times 2 \times b_p$	$\frac{b_p}{T \times b_t}$

As illustrated in Figure 7, this property becomes increasingly advantageous in the long-context regime. Due to the context-length invariance, the proportion of ADKV metadata remains marginal, typically below 2%, relative to the compressed KV cache when the context length exceeds 1K tokens. This highlights that ADKV achieves strong compression efficiency while introducing only minimal auxiliary memory overhead, making it particularly suitable for scalable long-context inference.

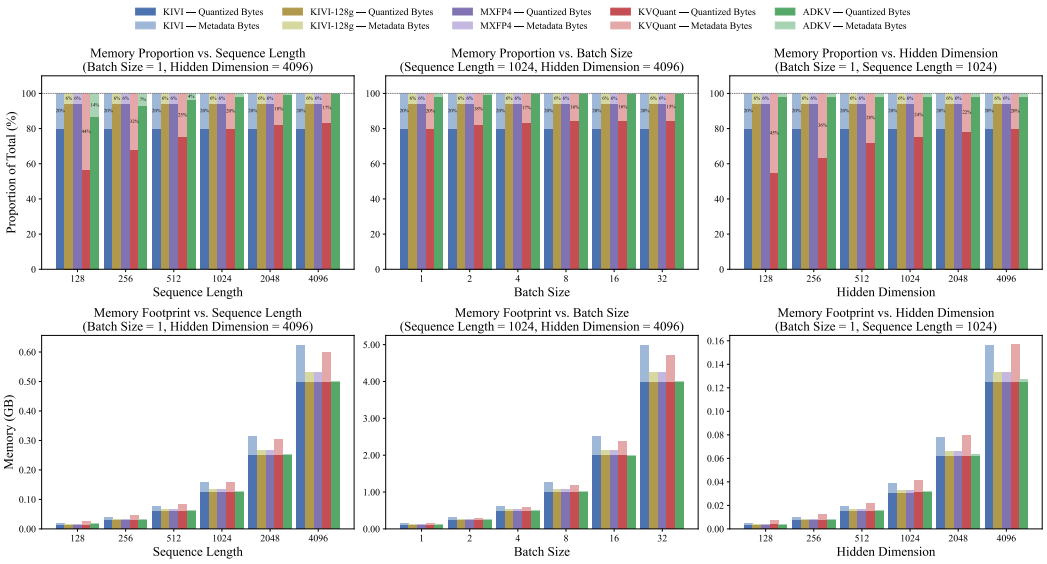


Figure 7: Memory footprint (absolute and proportion) comparison of KIVI (4-bit), KIVI-128g (4-bit), MXFP4, ADKV (4-bit), and KVQuant (4-bit).

H Fused ADKV-GEMV CUDA Kernel

In this section, we provide details of our fused ADKV-GEMV CUDA kernel, which fuses ADKV dequantization with matrix-vector multiplication. In addition, we report a comprehensive performance analysis of the fused ADKV-GEMV CUDA kernel.

H.1 Implementation

Algorithm 2 presents `gemv_adkv_qk`, a CUDA kernel that computes the query-key attention logits $\boldsymbol{s} = \boldsymbol{q}\boldsymbol{K}^\top$ under the ADKV quantization method. The kernel is designed for autoregressive inference, where the key cache grows token-by-token, and memory bandwidth is the dominant bottleneck. To

mitigate this, each key vector is stored in a 4-bit nibble-packed format, reducing the memory footprint of the KV cache by $4\times$ relative to a 16-bit baseline.

Parallelization Strategy. A single CUDA warp of $W = 32$ threads is assigned to each (b, h) pair, where b indexes the batch dimension and h indexes the attention head. Within a warp, the d_h-dimensional channel axis is partitioned evenly so that each thread owns $C_t = d_h/W$ channels. All per-channel parameters: zero point zp, scaling factor sc, ADKV tracking rates $alpha$, $beta$, quantization target tgt, and query value $qval$, are loaded from global memory into registers during the initial setup phase (Phase 1). Keeping these states in registers throughout the entire token loop avoids repeated global-memory traffic and enables the subsequent computation to proceed at near-peak arithmetic throughput.

Token Scan and Dequantization. Phase 2 performs a serial scan over all T cached tokens. For each token t, every thread independently unpacks the relevant int4 nibble for each of its C_t channels. Two tokens are co-stored in a single byte: the low nibble encodes an even-indexed token, and the high nibble encodes the subsequent odd-indexed token. After nibble extraction, a bias of 8 is subtracted to recover a signed integer in $[-8, 7]$, which is then dequantized to floating point via the mapping $k_{\text{deq}} = q_{\text{int}} \cdot sc + zp$. Each thread accumulates the partial dot product $\sum_{c \in \text{owned}} q_c \cdot k_{\text{deq},c}$ into a register scalar, and a warp-level butterfly reduction (`WarpReduce`) subsequently collapses the 32 partial sums into the final logit s_t, which is written to global memory by lane 0.

State Update. A distinguishing feature of this kernel is that the quantization parameters (zp, sc) are not static. They are updated in-place after each token is processed. Specifically, the zero point tracks a running mean of the dequantized key activations via the zero point tracking rate α, while the scaling factor adapts to the observed magnitude of the quantized value relative to a learned target T, modulated by β and clamped to the interval $[s_{\min}, s_{\max}]$ to prevent divergence. Because this recurrence is inherently sequential and all states are held in registers, it introduces no additional memory traffic and imposes only a handful of floating-point operations per channel and per token.

Complexity and Memory Access Pattern. The computational work per (b, h) pair is $\mathcal{O}(T \cdot d_h)$ floating-point operations, identical to a standard GEMV. Global memory reads amount to d_h floats for the query, $5d_h$ floats for the ADKV parameter tensors (loaded once), and $\lceil T/2 \rceil \cdot d_h$ bytes for the nibble-packed key cache, a $4\times$ reduction in key-cache bandwidth versus FP16 storage. The output tensor requires T float writes per (b, h) pair. This access pattern is well-suited to the memory hierarchy of modern GPUs, as the register-resident recurrence state eliminates shared-memory usage entirely and the sequential token scan issues coalesced loads across the warp's d_h-channel slice.

Attention Output. For computing the attention output $\boldsymbol{o} = \boldsymbol{aV}$, the `gemv_adkv_av` kernel follows the same strategy as `gemv_adkv_qk`. Hence, we omit the implementation details.

H.2 Performance Analysis

Figure 8 reports end-to-end throughput (requests/s over 32 layers, flattened hidden dimension $d_h = 4096$) as a function of sequence length T across eight batch sizes, comparing the FP16 baseline, KIVI (4-bit), KIVI-128g (4-bit), and the proposed ADKV (4-bit) kernel. Three distinct performance regimes emerge as the batch size increases.

Small-Batch Regime ($B \in \{1, 4\}$): quantization overhead outweighs bandwidth savings. At batch size $B = 1$, the FP16 baseline achieves approximately $1,907$ req/s at $T = 128$ and degrades gracefully to ~ 351 req/s at $T = 2048$, consistently outperforming both KIVI ($\sim 1,052$ and ~ 335 req/s, respectively) and ADKV (~ 449 and ~ 77 req/s). The same ordering, FP16 $>$ KIVI $>$ ADKV, persists for $B = 4$, with throughput gaps narrowing only modestly. This behavior is attributable to the fact that, at small batch sizes, the working set of the KV cache fits comfortably within the GPU's L2 cache or is serviced by a small number of memory transactions, so the kernel is compute-bound rather than memory-bound. Under these conditions, the overhead of nibble unpacking and dequantization in the quantized kernels (operations absent from the FP16 path) dominates, erasing the theoretical $4\times$ bandwidth reduction. Furthermore, ADKV's serial per-token recurrence (the ADKV state update in Phase 2 of Algorithm 2) prevents token-level parallelism that KIVI can still partially exploit, placing ADKV at an additional disadvantage in this regime.

Algorithm 2: CUDA Kernel `gemv_adkv_qk`: Attention Logits $\boldsymbol{s} = \boldsymbol{q}\boldsymbol{K}^\top$ with ADKV Quantization

Input: $\boldsymbol{q} \in \mathbb{R}^{B\times H\times d_h}$: query vectors (float16)
$\boldsymbol{K} \in \mathbb{Z}_4^{B\times H\times \lceil T/2\rceil \times d_h}$: nibble-packed int4 keys
$\boldsymbol{k}_{\text{zp}}, \boldsymbol{k}_{\text{s}}, \boldsymbol{\alpha}, \boldsymbol{\beta}, \boldsymbol{T} \in \mathbb{R}^{H\times d_h}$: channel-wise ADKV parameters
H: number of heads; T: sequence length; d_h: head dimension
$s_{\min}$, $s_{\max}$: scaling clamp bounds
Output: $\boldsymbol{s} \in \mathbb{R}^{B\times H\times T}$: attention logit scores

// Thread indexing — one warp ($W = 32$ threads) per block; one block per $(b,\ h)$ pair
$lane \leftarrow$ `threadIdx.x` *// $lane \in [0,\ W)$*
$bx \leftarrow$ `blockIdx.x` *// Linearized $(b,\ h)$ index*
$h \leftarrow bx \bmod H$ *// Head index*

// — Phase 1 — Load channel-wise parameters & query values into registers —
// $C_t = d_h/W$ channels per thread
for $ci \leftarrow 0$ **to** $C_t - 1$ **do**
  $c \leftarrow ci \cdot W + lane$ *// Global channel index*
  $pi \leftarrow h \cdot d_h + c$ *// Index into $[H, d_h]$ parameter arrays*
  $zp[ci] \leftarrow \boldsymbol{k}_{\text{zp}}[pi]$
  $sc[ci] \leftarrow \boldsymbol{k}_{\text{s}}[pi]$
  $alpha[ci] \leftarrow \boldsymbol{\alpha}[pi]$
  $beta[ci] \leftarrow \boldsymbol{\beta}[pi]$
  $tgt[ci] \leftarrow \boldsymbol{T}[pi]$
  $qval[ci] \leftarrow \boldsymbol{q}[bx \cdot d_h + c]$

// — Phase 2 — Serial token scan with warp-parallel dot products —
$T_{\text{pk}} \leftarrow \lceil T/2 \rceil$ *// Bytes per channel in nibble layout*
$\hat{\boldsymbol{K}} \leftarrow \boldsymbol{K} + bx \cdot T_{\text{pk}} \cdot d_h$ *// Base pointer for this (b, h)*
for $t \leftarrow 0$ **to** $T - 1$ **do**
  // Byte row = $\lfloor t/2 \rfloor$
  $row \leftarrow t \gg 1$
  // 0: low nibble (even t); 1: high nibble (odd t)
  $odd \leftarrow t\ \&\ 1$
  $local_sum \leftarrow 0.0$
  // Unrolled; all 32 lanes execute in lock-step
  for $ci \leftarrow 0$ **to** $C_t - 1$ **do**
    $c \leftarrow ci \cdot W + lane$
    // Packed byte: tokens $2\,row$ and $2\,row + 1$
    $p \leftarrow \hat{\boldsymbol{K}}[row \cdot d_h + c]$
    // Extract nibble $\Rightarrow$ signed int4 $\in [-8,\ 7]$
    if $odd = 1$ **then**
      $q_{\text{int}} \leftarrow (p \gg 4) - 8$ *// High nibble*
    else
      $q_{\text{int}} \leftarrow (p \&$ `0x0F`$) - 8$ *// Low nibble*
    $q_{\text{f}} \leftarrow \text{float}(q_{\text{int}})$
    // Dequantize
    $k_{\text{deq}} \leftarrow q_{\text{f}} \cdot sc[ci] + zp[ci]$
    // Partial dot-product
    $local_sum \mathrel{+}= qval[ci] \cdot k_{\text{deq}}$
    // ADKV state update
    $zp[ci] \leftarrow zp[ci] + alpha[ci] \cdot q_{\text{f}} \cdot sc[ci]$
    $sc[ci] \leftarrow \text{clamp}\big(sc[ci] \cdot \big(1 + beta[ci] \cdot (|q_{\text{f}}| - tgt[ci])\big), s_{\min}, s_{\max}\big)$
  $local_sum \leftarrow \text{WarpReduce}(local_sum)$ *// Butterfly reduction across 32 lanes*
  if $lane = 0$ **then**
    $\boldsymbol{s}[bx \cdot T + t] \leftarrow local_sum$

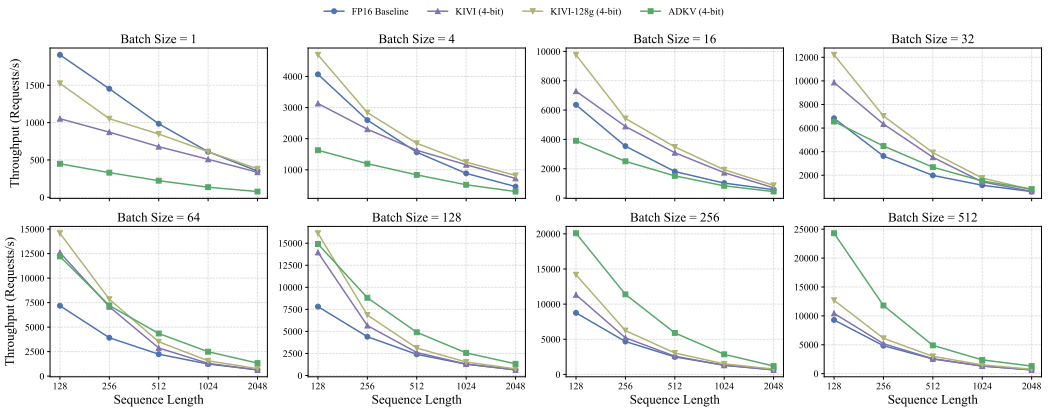


Figure 8: Throughput (requests/s, 32 layers, $d_h = 4096$) as a function of sequence length across eight batch sizes for four methods: FP16 baseline, KIVI (4-bit), KIVI-128g (4-bit), and ADKV (4-bit). Each sub-figure corresponds to a fixed batch size $B \in \{1, 4, 16, 32, 64, 128, 256, 512\}$.

Medium-Batch Regime ($B \in \{16, 32\}$): KIVI achieves peak relative advantage. As the batch size grows to 16 and 32, memory bandwidth pressure intensifies, and the gap between the FP16 baseline and the quantized methods narrows substantially. At $B = 32$, $T = 128$, KIVI delivers $\sim 9,864$ req/s, slightly surpassing ADKV ($\sim 6,554$ req/s) and the FP16 baseline ($\sim 6,829$ req/s). At longer sequences ($T = 512$), KIVI ($\sim 3,525$ req/s) maintains a visible lead over ADKV ($\sim 2,682$ req/s). The key factor in this regime is token-level parallelism. KIVI stores quantization metadata (scaling factors and zero points) at the group or channel granularity in a static fashion, allowing independent threads to dequantize arbitrary tokens in parallel with no sequential data dependencies. ADKV, by contrast, enforces a strict causal ordering: the quantization states (zp_t, sc_t) at token t depend on the decoded value of token $t-1$ (see the recurrence in Algorithm 2), making the token loop inherently serial. At $B \in \{16, 32\}$, the batch dimension alone cannot fully saturate the GPU's streaming multiprocessors, so the additional parallelism KIVI offers across the token dimension translates into measurable throughput gains.

Large-Batch Regime ($B \in \{64, 128, 256, 512\}$): ADKV becomes the dominant method. The performance hierarchy inverts decisively once the batch size reaches 64. At $B = 64$, $T = 512$, ADKV delivers $\sim 4,350$ req/s, surpassing both the FP16 baseline ($\sim 2,240$ req/s) and KIVI ($\sim 2,872$ req/s). The advantage widens with batch size: at $B = 256$, $T = 128$, ADKV achieves $\sim 20,084$ req/s versus $\sim 11,322$ req/s for KIVI and $\sim 8,758$ req/s for FP16. At $B = 512$, $T = 128$, the corresponding figures are approximately $24,299$, $10,431$, and $9,290$ req/s. Two complementary mechanisms explain this reversal. First, at large batch sizes, the GPU's warp schedulers are fully occupied by the $B \cdot H$ independent block instances, each processing one (b, h) pair. The serial token loop within each block is thus entirely hidden by inter-block parallelism, neutralizing ADKV's sequential decoding bottleneck. Second, and more critically, ADKV's adaptive method eliminates the need to store and transfer group-wise metadata alongside the quantized KV cache. KIVI must read a separate scaling factor and zero point entry for every quantization group, incurring additional global memory traffic proportional to T. ADKV encodes this information implicitly in the recurrent state, which lives entirely in registers; the only off-chip data movement is the nibble-packed key and value bytes themselves. At larger batch sizes, the kernel is firmly in the memory-bound regime, so this low-metadata property yields a substantial bandwidth advantage that translates directly into higher throughput. Taken together, these results suggest that ADKV is the preferred quantization method for high-throughput production inference services, while KIVI remains competitive for interactive, low-latency workloads with small batch sizes.

I Compatibility with Paged-Serving and Distributed KV Cache Systems

Paged-Serving System. PagedAttention [21] and similar paged KV management mechanisms do not fundamentally conflict with ADKV. PagedAttention mainly changes how KV cache memory is allocated, stored, and accessed, but it does not impose a specific computation order over KV

blocks. ADKV operates on the logical token sequence rather than the physical memory layout of the KV cache. In practical decoding systems, such as vLLM [21] and SGLang [46], KV cache blocks can still be traversed in logical token order during decoding, despite being physically paged [21]. Therefore, the temporal states maintained by ADKV can naturally propagate across block boundaries. Although paged KV management introduces non-contiguous memory layouts, decoding remains memory-bound due to dominant KV activation and metadata accesses. Since ADKV reduces metadata footprint and bandwidth requirements without depending on physical memory layout, it is compatible with paged KV cache systems.

Distributed KV Cache System. Distributed KV cache serving across multiple nodes/GPUs represents a more challenging scenario for ADKV. In such systems, a single request's KV cache may be sharded across different nodes/GPUs, preventing a straightforward global temporal state propagation. To support this setting, ADKV can maintain lightweight state checkpoints for each node/GPU, where each node/GPU stores the initial state required to continue adaptive quantization locally. ADKV can preserve such parallelism through lightweight state checkpoints.

Although this introduces minor metadata overhead, the number of nodes/GPUs is typically much smaller than the sequence length, making the checkpoint cost negligible compared with the KV cache size. Each shard only needs to store the ADKV state at its boundary. This can enable local continuation of state evolution without requiring global sequential traversal. For example, for Llama-2 7B, the checkpoint size is 32KB per layer and per request. Even with 8 KV shards (nodes), the total checkpoint metadata is only 8MB. In comparison, a 32K-token INT4 KV cache requires 4GB, resulting in less than 0.2% additional memory overhead. In addition, the state checkpoints introduce only negligible performance overhead. With 8 nodes, maintaining ADKV state checkpoints requires only 8 point-to-point transfers of boundary checkpoint states, with a total communication volume of 8MB throughout decoding.

J Ablations

J.1 Experimental Setup

All ablation experiments measure the quality of the ADKV quantization method using attention output MSE as the evaluation metric. Specifically, during the attention stage, each attention layer independently computes two attention outputs: one using the full-precision (FP16) key and value activations, and one using the quantized key and value activations after the ADKV online update step. The MSE between these two outputs is accumulated across all attention layers and all forward passes. All experiments reporting MSE are conducted using WikiText-2 [27] as input. A lower MSE indicates that the quantized KV cache introduces less distortion in the attention computation, implying higher fidelity to the full-precision baseline.

J.2 Component Ablation

Objective: This experiment quantifies the individual contribution of the two adaptive tracking mechanisms, zero point tracking and scaling factor tracking, to the overall quantization quality.

Table 6: Component Ablation Results

Variant	Zero Point Tracking (α)	Scaling Factor Tracking (β)	MSE ($\times 10^{-3}$) $\downarrow$
ADKV (full)	✓	✓	0.70
ADKV (no zero point tracking)	✗ ($\alpha \equiv \alpha_0$)	✓	8.25
ADKV (no scaling factor tracking)	✓	✗ ($\beta \equiv \beta_0$)	3.29
ADKV (no zero point & scaling factor tracking)	✗ ($\alpha \equiv \alpha_0$)	✗ ($\beta \equiv \beta_0$)	6.56

As shown in Table 6, disabling zero point tracking raises the MSE by $11.8\times$, the largest single-component degradation observed. Disabling scaling factor tracking raises the MSE by $4.7\times$. Interestingly, disabling both components simultaneously yields a slightly lower MSE (6.56×10^{-3}) than disabling zero point tracking alone (8.25×10^{-3}), suggesting a degree of interaction between the

two mechanisms: when the zero point is held fixed at its initial value, the scaling adaptation may partially overfit to the resulting residual distribution in a way that compounds rather than mitigates the distortion.

Both adaptive components are essential for high-fidelity quantization. Zero point tracking is the dominant contributor, reflecting the non-stationary nature of the KV cache distribution across token positions: the mean of each KV channel drifts significantly as context grows, and a fixed zero point fails to track this drift. Scaling factor tracking provides a complementary benefit by adapting the quantization resolution to the instantaneous spread of the residual signal. The full ADKV system, combining both mechanisms, achieves a $9.4\times$ reduction in attention output MSE relative to the no-tracking baseline.

J.3 Update Frequency Ablation

Objective: We evaluate the sensitivity of quantization quality to the update frequency F, i.e., the number of consecutive tokens processed before the online states (ZP_t, S_t) are updated.

Table 7: Update Frequency Ablation Results

Frequency F	MSE ($\times 10^{-3}$) $\downarrow$
1 (ADKV)	0.70
16	0.88
32	1.11
64	1.51
128	2.30

As shown in Table 7, MSE increases monotonically and approximately log-linearly with update frequency. Doubling the update frequency from 1 to 16 increases the MSE by 26%. Doubling again to 32, 64, and 128 produces cumulative increases of 59%, 116%, and 229% relative to the baseline, respectively.

These results demonstrate that fine-grained and per-token state updates ($F = 1$) are essential for achieving low-distortion ADKV quantization, and that quantization quality degrades systematically as the update granularity coarsens. The underlying cause is zero point drift in the KV cache: the zero point of each channel shifts continuously across token positions due to the evolving attention context. With $F = 1$, the states (ZP_t, S_t) are refreshed after every token, closely tracking this drift and keeping the residual well-centered within the quantization range. When F is increased, the states remain frozen for F consecutive tokens, during which the true channel mean and magnitude may drift substantially. The resulting misaligned residuals fall further from zero and are more likely to saturate the fixed quantization grid, leading to larger reconstruction errors and, consequently, higher attention output MSE. This sensitivity to update frequency directly motivates the per-token adaptive filter design at the core of ADKV.

J.4 Calibration Ablation

Objective: We assess the contribution of the SGD-based calibration to the final quantization quality.

Table 8: Calibration Ablation Results

Calibration	MSE ($\times 10^{-3}$) $\downarrow$
✓	0.70
✗ (default initialization)	3.07

As shown in Table 8, removing offline calibration increases the MSE by $4.4\times$. Without calibration, all five channel-wise parameters (ZP_0, S_0, α, β, T) retain their default initializations: $ZP_0 = 0$, $S_0 = 1$, $\alpha = \text{sigmoid}(0) \approx 0.5$, $\beta \approx 0$, and $T = Q_{\text{U}}/2 = 3.5$, following the initialization in Table 4.

Offline calibration provides a significant and consistent improvement in quantization quality. The calibration framework jointly optimizes all five parameters using representative KV cache sequences sampled from the target distribution, with STE/LSQ gradients for the scaling factor and TBPTT for temporal credit assignment. This allows the model to learn, for each channel, an appropriate initial zero point aligned with the typical magnitude of that channel, an initial scaling factor commensurate with its typical residual variance, and adaptive coefficients (α, β, T) tuned to the temporal dynamics of that channel. The $4.4\times$ MSE improvement demonstrates that these channel-wise statistics are highly heterogeneous and not well approximated by a uniform default initialization.

K Experiment Details

K.1 Generation and Long-Context Accuracy

Generation Task Evaluation. We use the `lm-evaluation-harness` framework to benchmark all methods. The evaluation tasks include `coqa` (EM), `truthfulqa_gen` (BLEU-acc), and `gsm8k` (flexible-extract exact match). We evaluate models `meta-llama/Llama-2-7b-hf`, `meta-llama/Llama-2-13b-hf`, `tiiuae/Falcon3-7B-Base`, and `mistralai/Mistral-7B-v0.1`. Unless otherwise specified, all results are obtained using default configurations.

To evaluate ADKV, we perform calibration using the `wikimedia/wikipedia` dataset, with 128 samples and 4096 tokens per sample. The calibration is conducted for 100 epochs, with TBPTT length set to 256. We use the Adam optimizer with a learning rate of 0.02.

Table 9: Long-Context Accuracy Results (Part I)

Model	Method	Qasper ↑	QMSum ↑	TREC ↑	TriviaQA ↑	MultiNews ↑	SAMSum ↑	LCC ↑	RepoB-P ↑
Llama-2 7B 32K	FP16	10.53	17.18	71.00	87.79	21.96	43.90	62.08	59.86
	RTN	10.26	14.48	68.50	75.08	19.55	34.81	49.91	46.93
	KIVI-128g	10.14	17.13	71.00	87.62	21.62	43.97	61.22	58.50
	ADKV (Ours)	10.35	17.16	71.00	87.49	21.32	43.98	61.62	58.98
Llama-3 8B 1048K	FP16	27.84	25.13	71.00	87.70	28.38	42.42	38.34	38.45
	RTN	26.45	23.91	70.00	84.26	28.01	41.14	30.84	33.37
	KIVI-128g	27.16	24.53	71.00	87.13	28.04	41.72	35.21	35.14
	ADKV (Ours)	27.52	24.63	72.00	86.33	28.31	41.69	35.32	35.35
Llama-2 13B	FP16	9.28	21.45	70.00	87.37	15.96	43.47	66.64	56.45
	RTN	6.80	17.57	61.00	72.12	13.40	41.26	61.41	51.46
	KIVI-128g	8.73	21.23	69.00	86.20	15.21	44.09	66.20	57.06
	ADKV (Ours)	8.81	21.48	70.00	87.17	15.53	43.51	65.94	57.10
Falcon3 7B	FP16	12.33	23.37	77.00	89.38	23.49	43.77	68.29	62.10
	RTN	12.11	23.19	73.50	89.18	22.34	41.74	66.97	61.68
	KIVI-128g	12.19	23.45	76.00	89.25	22.22	43.22	67.45	62.59
	ADKV (Ours)	12.03	22.81	76.50	89.40	22.72	43.25	67.79	62.12
Mistral 7B	FP16	8.07	19.16	66.50	89.21	13.83	41.24	69.31	61.48
	RTN	7.43	19.34	64.50	86.63	13.22	40.87	66.66	58.44
	KIVI-128g	7.55	19.03	65.50	89.10	13.62	40.95	69.36	60.21
	ADKV (Ours)	8.03	19.25	65.50	88.94	14.24	39.76	69.64	60.27

Table 10: Long-Context Accuracy Results (Part II)

Model	Method	NarrativeQA ↑	MultiFieldQA ↑	HotpotQA ↑	2WikiMQA ↑	Musique ↑	GovReport ↑	PassageCount ↑	PassageRetrieval ↑	Avg. ↑
Llama-2 7B 32K	FP16	15.28	33.43	12.36	12.51	6.20	29.42	1.50	24.50	31.84
	RTN	4.99	22.22	8.55	11.45	4.60	19.28	1.55	2.12	24.64
	KIVI-128g	14.81	32.13	12.83	12.64	6.38	28.78	1.50	20.16	31.28
	ADKV (Ours)	16.67	34.26	11.79	12.34	6.09	27.42	0.75	19.50	31.30
Llama-3 8B 1048K	FP16	22.74	52.45	32.00	21.00	14.05	34.38	5.50	90.50	39.49
	RTN	20.64	45.41	25.93	20.09	9.89	33.51	4.50	35.00	33.31
	KIVI-128g	22.88	51.56	33.89	22.49	13.38	34.12	6.00	77.50	38.23
	ADKV (Ours)	23.07	51.83	32.45	21.87	14.16	33.94	7.00	76.50	38.25
Llama-2 13B	FP16	12.91	19.66	9.33	11.00	5.69	19.17	1.75	5.56	28.48
	RTN	6.89	17.01	8.77	10.41	4.40	10.71	1.10	5.18	24.34
	KIVI-128g	13.02	18.98	9.22	10.70	5.24	19.19	3.12	6.30	28.34
	ADKV (Ours)	12.93	18.28	9.74	10.71	5.53	19.02	2.00	6.36	28.38
Falcon3 7B	FP16	22.46	34.16	11.07	12.64	6.72	33.56	2.93	44.71	35.50
	RTN	18.82	35.28	10.63	11.22	6.11	31.93	1.17	35.62	33.84
	KIVI-128g	22.12	34.51	10.22	13.46	6.07	32.75	1.71	37.31	34.66
	ADKV (Ours)	22.33	35.15	9.50	11.86	5.45	33.20	2.71	44.83	35.10
Mistral 7B	FP16	6.16	26.09	9.68	11.20	4.49	20.70	1.00	14.18	28.89
	RTN	5.34	22.08	8.78	10.90	4.79	17.75	0.23	7.63	27.16
	KIVI-128g	4.25	25.21	10.13	11.08	4.52	20.27	1.50	13.68	28.50
	ADKV (Ours)	8.49	26.03	9.63	11.64	3.76	19.32	1.00	13.26	28.67

Long-Context Task Evaluation. LongBench is composed of 6 major categories and 16 different tasks, covering key long-text application scenarios such as single-document QA, multi-document QA, summarization, few-shot learning, synthetic tasks, and code completion. We evaluate the long-context models `togethercomputer/LLaMA-2-7B-32K` and `gradientai/Llama-3-8B-Instruct-Gradient-1048k`. We use the evaluation scripts provided by the LongBench GitHub repository. We follow the same calibration strategy as in Appendix K.1.

Here, we report the full results on LongBench (Tables 9 and 10), extending Table 2.

K.2 Accuracy Comparison with Additional Baselines

To further evaluate our method against additional baselines, we evaluate KIVI-32g, KVQuant, MXFP4, CQ [42], ZipCache [17], and PolarQuant [15] on the WinoGrande [33], PIQA [5], and ARC-C [6] at matched bits per activation (BPA). The corresponding accuracy results are reported in Table 11.

Table 11: Accuracy Results with Additional Baselines

Model	Method	BPA	WinoGrande ↑	PIQA ↑	ARC-C ↑
Llama-2 7B	FP16	16	68.90	78.07	43.43
	KVQuant-4b	4	67.96	77.86	42.75
	KVQuant-4b+1%	4.32	68.67	78.07	43.17
	KIVI-32g	5	68.65	78.07	43.17
	MXFP4	4.25	68.27	77.91	42.32
	CQ-2c8b	4	68.27	77.91	43.34
	ZipCache-4b	4	67.60	77.91	42.15
	PolarQuant	3.88	66.69	77.22	42.92
	ADKV-4b	4	68.75	77.42	42.41
Llama-2 13B	FP16	16	71.98	79.16	48.29
	KVQuant-4b	4	71.59	78.94	46.67
	KVQuant-4b+1%	4.32	72.30	79.27	47.87
	KIVI-32g	5	72.15	79.02	46.67
	MXFP4	4.25	71.84	78.78	46.66
	CQ-2c8b	4	72.53	78.62	47.78
	ZipCache-4b	4	72.22	78.78	45.14
	PolarQuant	3.88	71.38	78.07	46.93
	ADKV-4b	4	72.45	78.68	46.93

We highlight the key differences between ADKV and CQ, ZipCache, and PolarQuant as follows:

CQ. Although CQ and ADKV both aim to reduce quantization metadata overhead, they adopt different approaches. CQ removes context-length-dependent metadata through shared learned codebooks, but introduces additional runtime operations such as codebook lookup and indexing. In contrast, ADKV eliminates frequent group-wise metadata accesses by using lightweight in-register state updates, a design that avoids the runtime overhead associated with codebook lookup and indexing.

ZipCache. ZipCache reduces quantization metadata overhead through channel-separable per-token quantization, which can be viewed as increasing the sharing granularity of quantization metadata. While it achieves competitive accuracy under 4-bit quantization, its accuracy degrades under 2-bit quantization. In contrast, ADKV maintains reasonable accuracy at 2-bit precision through adaptive delta quantization (Table 12).

PolarQuant. PolarQuant and ADKV share the objective of reducing metadata overhead, but they achieve this goal through fundamentally different mechanisms. PolarQuant applies random preconditioning and a recursive polar transformation, quantizes the resulting angles with codebooks, and retains a full-precision radius for each coordinate block. Therefore, it eliminates conventional zero point/scaling factor metadata but still requires radius storage and a specialized polar-domain encoding and reconstruction pipeline. In contrast, ADKV remains in the Cartesian domain and leverages the temporal trackability of KV activations to recursively reconstruct channel-wise zero points and scaling factors as ADKV states. This design eliminates group-wise metadata overhead

while only requiring lightweight in-register multiply-add quantization. Thus, PolarQuant is primarily a representation transformation method, whereas ADKV is an adaptive state evolution method.

K.3 Accuracy Results for 2-Bit ADKV

ADKV does not impose any constraints on the choice of quantization bit-width. To further evaluate its accuracy under more aggressive quantization, we conduct additional experiments using 2-bit quantization. The corresponding accuracy results are reported in Table 12.

Table 12: Accuracy Results for 2-Bit ADKV

Model	Method	BPA	WinoGrande ↑	PIQA ↑	ARC-C ↑
Llama-2 7B	FP16	16	68.90	78.07	43.43
	KVQuant-2b	2	51.70	63.38	22.44
	KVQuant-2b+1%	2.32	67.64	76.60	41.47
	KIVI-2b	3	65.90	76.06	40.36
	CQ-4c9b	2.26	67.40	77.09	41.64
	CQ-4c8b	2	66.45	76.12	39.93
	ZipCache-2b	2	49.01	54.68	24.66
	ADKV-2b	2	65.68	76.01	41.04
Llama-2 13B	FP16	16	71.98	79.16	48.29
	KVQuant-2b	2	51.30	65.40	24.66
	KVQuant-2b+1%	2.32	70.17	78.51	44.97
	KIVI-2b	3	68.03	78.02	45.65
	CQ-4c9b	2.26	71.67	77.31	44.97
	CQ-4c8b	2	69.06	77.42	44.11
	ZipCache-2b	2	49.49	52.45	21.50
	ADKV-2b	2	68.27	77.86	44.54

L Limitations

While ADKV demonstrates strong performance across a range of models and benchmarks, several directions remain open for future investigation. First, the current calibration procedure requires a representative offline dataset, which may limit applicability in fully online or privacy-sensitive settings. Exploring online calibration strategies is a promising direction. Second, ADKV currently focuses on uniform quantization. Combining it with non-uniform or mixed-precision schemes may yield further accuracy improvements, particularly for channels with irregular activation distributions. Finally, extending ADKV to support industrial LLM inference frameworks, such as vLLM and SGLang, under segmented KV cache management remains an interesting research direction.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The main claims in the abstract and introduction (e.g., memory footprint reduction and throughput) are justified in the evaluation in Section 4.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: We discuss the limitations in Appendix L.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [Yes]

Justification: We provide a comprehensive analysis and proof of our method, including gradient analysis and system analysis in Appendix D and Appendix E, respectively.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: We provide comprehensive implementations of our method, including the calibration algorithm and CUDA kernels in Appendix F and Appendix H, respectively. In addition, we provide all experiment details in Section 4 and Appendix K.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
     (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
     (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: We provide the complete code at https://github.com/LoveLonelyTime/ADKV, including the calibration framework, the fused ADKV-GEMV CUDA kernel, and the Llama model integrated with ADKV.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (https://neurips.cc/public/guides/CodeSubmissionPolicy) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (https://neurips.cc/public/guides/CodeSubmissionPolicy) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: We provide the comprehensive experiment details in Appendix K.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: All experiments use publicly available benchmark datasets and open-source evaluation scripts. The results show consistently low standard deviations, indicating stable performance with minimal variance across runs.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: We provide the hardware environment that we adopt in Section 4.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: We have reviewed the NeurIPS Code of Ethics and confirm that our work is fully aligned with its principles.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [N/A]

    Justification: This work focuses on improving KV cache quantization algorithms and related technical methods. It does not have any direct or significant societal impacts at present. Therefore, we do not include a discussion of potential positive or negative societal effects.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The paper does not release any models or datasets that present a high risk of misuse. Therefore, no additional safeguards are required.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: We use publicly available datasets and pretrained models, all of which are appropriately cited in the paper.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.

- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The paper releases new assets in the form of implementation code. The code and accompanying documentation are publicly available at `https://github.com/LoveLonelyTime/ADKV`.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: We do not conduct any crowdsourcing experiments or research with human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: We do not conduct any research with human subjects.

    Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

    Justification: LLMs do not affect any aspect of the methodology, scientific rigor, or originality of the research.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.